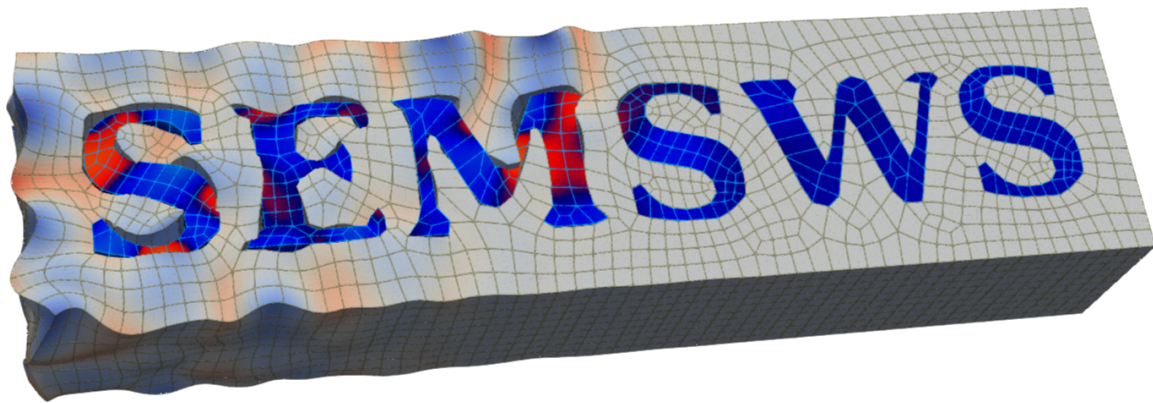




Spectral Element Method
Seismic Wave Simulator

User Manual

Version 0.1

May 12, 2026

Contents

1	Introduction	7
1.1	What is SEMSWS?	7
1.2	Supported Physical Models	7
1.3	Key Aspects of the Spectral Element Method	8
1.4	Structure of This Manual	8
2	Installation	9
2.1	Requirements	9
2.1.1	Required	9
2.1.2	Optional	9
2.1.3	Python packages for running pytest	9
2.2	Building the Dependencies	10
2.2.1	Building hypre	10
2.2.2	Building MFEM	10
2.3	Building SEMSWS	11
2.3.1	Basic build procedure	11
2.3.2	CMake options	12
2.3.3	GPU build	12
2.4	Build Configuration Examples	12
2.4.1	Local environment (WSL2/Ubuntu, CPU only)	12
2.4.2	Earth Simulator (NVIDIA A100 GPU)	13
2.4.3	LUMI (AMD MI250X GPU)	14
2.5	Installing with Spack	15
2.5.1	Advantages of the Spack route	15
2.5.2	Local environment (generic Linux)	15
2.5.3	Earth Simulator (NVIDIA A100 GPU)	16
2.5.4	Earth Simulator (CPU)	18
2.5.5	Second and subsequent sessions (ES)	19
2.5.6	Example ES job script (Spack build)	19
2.5.7	Running pytest in the Spack environment	20
2.5.8	Appendix: rationale for the ES Spack configuration	20
2.6	Verifying the Build	22
2.7	Installing GLVis (Recommended)	23
2.7.1	Building	23
2.7.2	Basic usage	24
3	Quick Start	25
3.1	Configuration File	25
3.1.1	Basic simulation settings	25
3.1.2	Mesh	25
3.1.3	Material	26

3.1.4	Boundary conditions	26
3.1.5	Source	26
3.1.6	Receivers	27
3.1.7	Wavefield output	27
3.1.8	Device settings	27
3.2	Running the Simulation	27
3.3	Checking the Results	28
3.3.1	Summary	28
3.3.2	Wavefield snapshots	29
3.3.3	Receiver waveforms	29
4	Input File Reference	31
4.1	YAML Format Basics	31
4.2	Strict-Key Validation	32
4.3	Overall Structure of the Configuration File	32
4.4	Root-Level Parameters	32
4.5	simulation section	33
4.5.1	Basic parameters	33
4.5.2	Time integration (time)	33
4.5.3	Simulation mode	33
4.5.4	Output settings (output)	34
4.5.5	Wavefield output (output.wavefield)	34
4.5.6	Material output (output.material)	35
4.5.7	Example settings for the simulation section	35
4.6	mesh section	36
4.6.1	Common parameters	36
4.6.2	Internal mesh (type: internal)	36
4.6.3	External mesh (type: external)	37
4.6.4	Pre-partitioned mesh (type: partitioned)	37
4.6.5	Example settings for the mesh section	37
4.7	material section	38
4.7.1	Material types and formats	38
4.7.2	format: constant	38
4.7.3	format: grid	38
4.7.4	format: by_attribute	39
4.7.5	format: by_attribute_mixed	39
4.7.6	format: adios2	40
4.7.7	Attenuation (attenuation)	40
4.7.8	type: coupled (fluid–solid coupling)	40
4.7.9	Model export	42
4.7.10	Example settings for the material section	42
4.8	boundary section	43
4.8.1	Absorbing boundary conditions (absorbing)	43
4.8.2	Dirichlet boundary conditions (dirichlet)	43
4.8.3	Example settings for the boundary section	44
4.9	sources section	44
4.9.1	Source mode and input format	44
4.9.2	Common source parameters (list)	44
4.9.3	Source type: force	44
4.9.4	Source type: pressure	45
4.9.5	Source type: moment_tensor	45
4.9.6	Wavelet (wavelet)	45

4.9.7	Observed data (observed) — inversion mode	46
4.9.8	Example settings for the sources section	46
4.10	receivers section	48
4.10.1	Recording type and output settings	48
4.10.2	Receiver line (line)	49
4.10.3	Example settings for the receivers section	49
4.10.4	External receiver file (receivers.file)	50
4.11	device section	51
4.11.1	Example settings for the device section	51
4.12	inversion section	51
4.13	HDF5 I/O Schema and Usage	51
4.13.1	Overview of the HDF5 v2.0 schema (common)	52
4.13.2	Input path: HDF5 -> SEMSWS	52
4.13.3	Output path: SEMSWS -> HDF5	56
5	Tools and Scripts	61
5.1	partition_mesh — Pre-partition a mesh	61
5.1.1	Usage	61
5.1.2	Options	61
5.1.3	Use in config.yaml	62
5.2	combine_mesh — Combine partitioned mesh	62
5.3	evaluate_mesh — Mesh quality evaluation	62
5.3.1	Usage	62
5.3.2	Options	62
5.3.3	Histograms produced	62
5.4	refine_mesh — Uniform mesh refinement and partitioning	63
5.4.1	Behavior of mesh loading and output	63
5.4.2	Usage	64
5.4.3	Options	64
5.5	Workflow example: Mesh evaluation and refinement	64
5.5.1	Problem setup	64
5.5.2	Step 1: Evaluating the initial mesh	65
5.5.3	Step 2: Uniform refinement	66
5.5.4	Step 3: Re-evaluating after refinement	66
5.5.5	Step 4: Running the simulation	67
5.6	Visualization scripts	67
5.6.1	plot_wavefield_2d.py — 2D wavefield visualization	67
5.6.2	plot_wavefield_3d.py — 3D cross-section wavefield visualization	67
5.6.3	plot_3d_slices.py — PyVista 3D visualization	68
5.6.4	plot_mesh_quality.py — Mesh quality histograms	69
6	Application Examples	71
6.1	visualization — Visualization demos	71
6.2	elastic_analytic — Analytical-solution benchmark	71
6.2.1	Problem setup	72
6.2.2	How to run	72
6.2.3	Visualization of the result	72
6.3	visco_elastic_3d_layered — 3D viscoelastic two-layer model	72
6.3.1	Included configurations	72
6.3.2	How to run	72
6.4	fun	73
6.4.1	ICM — 2D elastic wave	73

6.4.2	JAMSTEC — 2D acoustic wave	74
7	Performance	77
7.1	Test environment	77
7.2	Strong Scaling	77
7.3	Weak Scaling	78
7.4	GPU vs CPU	79
	Development Team	81

Chapter 1

Introduction

1.1 What is SEMSWS?

SEMSWS (Spectral Element Method – Seismic Wave Simulator) is a seismic wave propagation simulation solver based on the spectral element method (SEM). It is built on top of the finite element library MFEM¹. MFEM is an open-source library developed primarily at Lawrence Livermore National Laboratory. Many of the core capabilities of SEMSWS—high-order finite element spaces, support for diverse mesh formats, MPI-parallel assembly, and GPU/CPU portability via CUDA/HIP/OpenMP backends—are underpinned by MFEM. We gratefully acknowledge the MFEM development team. This enables execution on both CPUs and GPUs (CUDA/HIP) from a single source code.

SEM, a widely used numerical method for solving the seismic wave equation, employs high-order Lagrange interpolation on Gauss–Lobatto–Legendre (GLL) points. Its main advantages include efficient explicit time integration through mass matrix diagonalization, support for unstructured meshes, natural incorporation of free-surface boundary conditions via the weak formulation, and high-accuracy, low-dispersion solutions based on spectral convergence.

1.2 Supported Physical Models

SEMSWS supports the following wave equations in both 2D and 3D:

- **Isotropic Elastic Wave Equation** (Isotropic Elastic)
- **Isotropic Acoustic Wave Equation** (Isotropic Acoustic)
- **Viscoelastic / Viscoacoustic Attenuation** (Viscoelastic / Viscoacoustic)

Future updates are planned to add support for VTI, full anisotropy, and solid–fluid coupled domains.

The polynomial order can be set to any value; however, the GPU implementation imposes an upper limit due to shared memory constraints of the stiffness kernel.

Time integration uses the explicit Newmark scheme ($\beta = 0$, $\gamma = 1/2$, equivalent to the central difference method). This scheme is second-order accurate in time and conditionally stable under the CFL condition.

For absorbing boundary conditions, a sponge absorbing layer (Cerjan damping) is implemented. Perfectly matched layers (PML) are planned for future development.

Full Waveform Inversion (FWI) functionality is currently under development. It is partially supported only for 2D acoustic cases, and testing remains insufficient.

¹<https://mfem.org/>

1.3 Key Aspects of the Spectral Element Method

The main SEM parameter specified by the user in the configuration file is the **polynomial order** (`order`).

When `order` = N , $N + 1$ GLL points are placed along each edge of every element. For seismic wave simulations, `order` = 4–5 is typical.

The essential advantage of SEM is that using GLL points for both interpolation and quadrature yields a diagonal mass matrix. This allows the acceleration field to be computed by simple element-wise division, eliminating the need to solve a system of equations. The stiffness term $\mathbf{KU}$ is not assembled as a matrix but computed matrix-free through element-level operations, reducing both memory usage and computational cost.

1.4 Structure of This Manual

Chapter 2: Installation Describes the procedure from preparing the dependencies (hypre, MFEM) through the build process. Includes build configuration examples for various computing environments (local, ES, LUMI).

Chapter 3: Quick Start Walks through the complete workflow—from writing a configuration file to running a simulation and visualizing the results—using a 2D acoustic wave simulation as an example.

Chapter 4: Input File Reference Provides a comprehensive description of all options in `config.yaml`.

Chapter 5: Tools and Scripts Explains how to use the mesh partitioning, refinement, and quality evaluation tools, as well as the visualization scripts.

Chapter 6: Application Examples Presents example cases including analytical-solution benchmarks and 3D viscoelastic models.

Chapter 7: Performance Shows scaling results on ES and LUMI, along with GPU/CPU performance comparisons.

Chapter 2

Installation

2.1 Requirements

The following software is required to build SEMSWs.

2.1.1 Required

Table 2.1: Required dependencies

Software	Notes
C++17 compiler	<code>-std=c++17</code> support
CMake	Build system
MPI	Parallel computing
MFEM	Must be built with MPI enabled
ADIOS2	Parallel I/O (BP format)
HDF5	Waveform output
yaml-cpp	Fetches automatically by CMake (no manual install needed)

2.1.2 Optional

Table 2.2: Optional dependencies

Software	Notes
CUDA	NVIDIA GPU support
HIP	AMD GPU support

2.1.3 Python packages for running pytest

To run the test suite under `test/`, in addition to Python 3.10 or later, install the following packages via `pip`:

Table 2.3: Python packages required for pytest

Package	Required/Optional	Purpose
pytest	Required	Test runner
numpy	Required	Waveform comparison and numerical verification
h5py	Required	HDF5 I/O tests (e.g., <code>test/consistency/</code>)
pyyaml	Required	Reading YAML configuration files
matplotlib	Optional	Plotting tests and diagnostic figures
gmsh	Optional	Mesh generation tests for fluid-solid coupling (<code>test/coupling/</code>)

Bulk install example:

```
pip install pytest numpy h5py pyyaml matplotlib gmsh
```

The Python driver under `driver/` (`semsws-driver`) has its own dependencies. They are resolved automatically from `driver/pyproject.toml` via `pip install -e ./driver`.

2.2 Building the Dependencies

SEMSWS is built on top of the MFEM library, and MFEM in turn depends on hypre. These must be built with consistent precision and GPU backend settings.

2.2.1 Building hypre

CPU build

```
cd hypre/src
./configure --enable-single --with-MPI
make -j$(nproc)
```

For double precision, omit `--enable-single`.

CUDA build (NVIDIA GPU)

```
cd hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-cuda --enable-gpu-aware-mpi \
  CUDA_HOME=/path/to/cuda HYPRE_CUDA_SM=80
make -j$(nproc)
```

HIP build (AMD GPU)

```
cd hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-hip --enable-gpu-aware-mpi \
  CC=hipcc CXX=hipcc
make -j$(nproc)
```

`--enable-unified-memory` is generally recommended for GPU environments.

2.2.2 Building MFEM

MFEM is the finite element library at the core of SEMSW. Build it with CMake using the options below.

Example build (CPU, single precision)

```
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HDF5=ON \
  -DHDF5_DIR=/path/to/hdf5 \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=/path/to/hypre/src/hypre
cmake --build build -j$(nproc)
```

NETCDF is not strictly required, but it is needed to read meshes generated with tools such as CUBIT.

Precision consistency The precision settings of hypre (`--enable-single`) and MFEM (`MFEM_PRECISION`) must match. SEMSWs automatically inherits MFEM's precision setting, so as long as hypre and MFEM are consistent, no configuration on the SEMSWs side is required.

For details on building MFEM, refer to the official documentation¹.

2.3 Building SEMSWs

2.3.1 Basic build procedure

Run the following commands from the root of the source directory:

```
# 1. CMake configure
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DMFEM_DIR=/path/to/mfem/build

# 2. Build
cmake --build build -j$(nproc)

# 3. Install (places executables under CMAKE_INSTALL_PREFIX)
cmake --install build
```

MFEM_DIR should point to either the build or install directory of MFEM. Specifying `CMAKE_INSTALL_PREFIX` and running `cmake --install build` places `semsws` and the auxiliary tools under `<prefix>/bin/`, the static libraries under `<prefix>/lib/`, and the headers under `<prefix>/include/semsws/`.

¹<https://mfem.org/building/>

2.3.2 CMake options

Table 2.4: CMake options

Option	Default	Description
MFEM_DIR	—	Path to MFEM (required)
CMAKE_BUILD_TYPE	—	Release or Debug
CMAKE_INSTALL_PREFIX	/usr/local	Destination of <code>cmake --install</code> build
CMAKE_CXX_COMPILER	auto-detected	MPI compiler
CMAKE_CUDA_ARCHITECTURES	70;80	CUDA architecture
AMDGPU_TARGETS	gfx90a	AMD GPU architecture
SEM_USE_GPU_AWARE_MPI	OFF	Declares at build time that the MPI library being linked is GPU-aware (CUDA/ROCm capable). With <code>OFF</code> (default), buffers for fluid-solid coupling are staged through the host (safe with any MPI). Set <code>ON</code> only in environments verified to be GPU-aware, such as LUMI Cray MPICH (<code>MPICH_GPU_SUPPORT_ENABLED=1</code>) or CUDA-aware OpenMPI (an incorrect <code>ON</code> will crash inside <code>MPI_Isend</code>).

2.3.3 GPU build

If MFEM is built with CUDA or HIP, SEMSWs automatically becomes a GPU build.

2.4 Build Configuration Examples

The following sections show build procedures for each environment.

2.4.1 Local environment (WSL2/Ubuntu, CPU only)

Example configuration with single precision and CPU only.

```
# hypr
cd $HOME/program_ubuntu/hypr/src
./configure --enable-single --with-MPI
make -j$(nproc)

# MFEM
cd $HOME/program_ubuntu/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HDF5=ON -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$HOME/program_ubuntu/hypr/src/hypr
cmake --build build -j$(nproc)

# SEMSWs
cd $HOME/program_ubuntu/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Debug \
```

```

-DCMAKE_INSTALL_PREFIX=<install_dir> \
-DMFEM_DIR=$HOME/program_ubuntu/mfem/build
cmake --build build -j$(nproc)
cmake --install build

```

2.4.2 Earth Simulator (NVIDIA A100 GPU)

Example for the GPU partition (NVIDIA A100) of JAMSTEC's Earth Simulator. The NVIDIA HPC SDK 24.7 (nvc++/nvcc) is used as the compiler. A configuration verified to work as of April 2025 is the following: NVIDIA HPC SDK 24.7, CUDA 12.0.0, hypre 2.31.0, METIS 5.1.0, HDF5 1.10.5.

```

# Load modules
module load HDF5/1.10.5
module load NetCDF4/all
module load NVIDIAHPCSDK/24.7/nvhpc-hpcx-cuda12
module load METIS/5.1.0
module load CUDA/12.0.0

# hypre
cd $HOME/program_gpu/hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-cuda --enable-gpu-aware-mpi \
  CUDA_HOME=/opt/share/CUDA/12.0.0 HYPRE_CUDA_SM=80
make -j$(nproc)

# MFEM (using the hypre built above)
cd $HOME/program_gpu/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_CUDA=ON \
  -DMFEM_USE_HDF5=ON \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$HOME/program_gpu/hypre/src/hypre \
  -DMETIS_DIR=/opt/share/METIS/5.1.0
cmake --build build -j$(nproc)

# SEMSW
cd $HOME/program_gpu/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DCMAKE_CXX_STANDARD=17 \
  -DSEM_USE_GPU_AWARE_MPI=ON \
  -DMFEM_DIR=$HOME/program_gpu/mfem/build \
  -Dadios2_DIR=$HOME/program_cpu/ADIOS2/build
cmake --build build -j
cmake --install build

```

The only paths the SEMSWs cmake line needs are MFEM (MFEM_DIR) and ADIOS2 (adios2_DIR). hypre and METIS are linked transitively through MFEM and do not appear on the SEMSWs cmake line.

2.4.3 LUMI (AMD MI250X GPU)

Example for the LUMI-G partition of CSC Finland's LUMI supercomputer. The compiler is hipcc from ROCm 6.4.4, and the MPI implementation is Cray MPICH. A configuration verified to work as of April 2026 is the following: ROCm 6.4.4, Cray PE 25.09 (CCE 20.0.0), hypre 3.1.0, METIS 5.1.0 (cpeCray-25.09-idx32-fp32), cray-hdf5 1.14.3.7.

```
# Load modules
module load LUMI/25.09 partition/G
module load METIS/5.1.0-cpeCray-25.09-idx32-fp32
module load cray-hdf5/1.14.3.7

# hypre
cd $FD/program/hypr/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-hip --enable-gpu-aware-mpi \
  CC=hipcc CXX=hipcc \
  LDFLAGS="-L/opt/cray/pe/mpich/9.0.1/gtl/lib" \
  LIBS="-lmpi_gtl_hsa"
make -j$(nproc)

# MFEM (using the hypr built above)
cd $FD/program/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HIP=ON \
  -DAMDGPU_TARGETS=gfx90a \
  -DMFEM_USE_HDF5=ON \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$FD/program/hypr/src/hypr \
  -DHDF5_INCLUDE_DIRS=/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/include \
  -DHDF5_LIBRARIES="/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5_cpp.so" \
  /opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5.so"
cmake --build build -j$(nproc)

# SEMSWs
cd $FD/program/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DAMDGPU_TARGETS=gfx90a \
  -DSEM_USE_GPU_AWARE_MPI=ON \
  -DMFEM_DIR=$FD/program/mfem/build \
  -Dadios2_DIR=$FD/program/ADIOS2/build
cmake --build build -j
```

```
cmake --install build
```

The only paths the SEMSWS cmake line needs are MFEM (`MFEM_DIR`) and ADIOS2 (`adios2_DIR`). `hypre`, `METIS`, and `HDF5` are linked transitively through MFEM and do not appear on the SEMSWS cmake line. MPI jobs are submitted via `srun`.

2.5 Installing with Spack

The manual builds in the preceding sections require human management of version consistency across dependencies, which is tedious. The SEMSWS repository bundles Spack recipes under `spack-repo/` and a preconfigured site file for Earth Simulator under `site-config/es/`, allowing the full dependency build to be automated through Spack.

This section walks through the Spack install procedure command by command for two environments: a local Linux machine and Earth Simulator (hereafter ES). The rationale behind each command and flag is collected in §2.5.8 at the end.

Spack version requirement. The SEMSWS `package.py` uses the explicit language declarations (`depends_on("c", type="build") / depends_on("cxx", type="build")`) introduced in Spack v1.0, so **Spack 1.0 or later** is required. Confirmed to work with v1.1.x (stable) and v1.2.x (develop branch).

2.5.1 Advantages of the Spack route

- **Automatic dependency resolution:** version constraints for `hypre` / `MFEM` / `ADIOS2` / `HDF5` / `NetCDF` / `METIS` / `openblas` are centralized in the SEMSWS `package.py`.
- **Site-specific pitfalls absorbed:** the NVHPC prefix-layout bug, autotools inconsistencies specific to `nvc`, the single-precision FSAI bug in `cuSOLVER`, the CMake breakage in `hypre 3.x`—all of these are handled by shadow recipes on the SEMSWS side.
- **Reproducibility:** the pinned `spack.yaml` + `spack.lock` inside a `spack env` reproduce the full build configuration.

2.5.2 Local environment (generic Linux)

For a local Linux machine or a generic cluster that does not need ES-specific adjustments.

Step 1: Clone SEMSWS.

```
git clone https://github.com/SEMSWS/semsws.git
cd semsws
```

Step 2: Set up Spack itself. Skip if spack is already available. Otherwise:

```
git clone --depth=1 https://github.com/spack/spack.git ~/spack
source ~/spack/share/spack/setup-env.sh
```

The `source` line is required each time a new shell is opened; adding it to `~/.bashrc` is convenient.

Step 3: Register the SEMSWS Spack repo (once).

```
spack repo add ./spack-repo
```

This tells Spack about SEMSWS and its shadow recipes (`hypre`, `yaml-cpp`, etc.).

Step 4 (recommended): Register cmake as an external. Without this, Spack will build cmake from source (10+ minutes). Declare the system cmake (3.20 or newer) as an external:

```
mkdir -p ~/.spack
cat >> ~/.spack/packages.yaml <<'EOF'
packages:
  cmake:
    externals:
      - spec: cmake@3.28.0      # match the output of `cmake --version`
        prefix: /usr           # prefix returned by `which cmake`
        buildable: false
EOF
```

Declaring git and python as externals in the same way dramatically shortens the first-time build.

Step 5: Install.

```
spack install semsws@0.1.0
```

Dependencies (hypre / MFEM / ADIOS2 / yaml-cpp / others) are built automatically. To install the development version, use `semsws@main`.

Step 6: Use.

```
spack load semsws
which semsws      # /path/to/spack/opt/.../semsws-.../bin/semsws
mpirun -np 4 semsws --config config.yaml
```

2.5.3 Earth Simulator (NVIDIA A100 GPU)

The GPU partition of ES. NVHPC SDK 24.7 + HPC-X OpenMPI + CUDA 12.0.0 + single precision + GPU-aware MPI configuration.

Step 1: Set up the working directory and environment variable.

```
export WORK=/S/data01/G3506/${USER}/program_test # adjust to your own
path
mkdir -p $WORK
```

Everything (Spack itself, cache, the SEMSWs repository) lives under `WORK`. Avoid home because of its inode quota.

Step 2: Clone Spack (once).

```
git clone --depth=1 https://github.com/spack/spack.git $WORK/spack
```

`--depth=1` limits history to a single commit, making the clone fast.

Step 3: Clone SEMSWs.

```
cd $WORK
git clone https://github.com/SEMSWS/semsws.git
cd semsws
```

Step 4: Copy the GPU load script to home.

```
cp spack-repo/site-config/es/load_gpu.sh ~/load_gpu.sh
chmod +x ~/load_gpu.sh
```

This script bundles module load, environment variable setup, and Spack initialization. The contents are documented in §2.5.8.

Step 5: Load the environment.

```
source ~/load_gpu.sh
```

This loads the modules and makes the `spack` command available (it internally runs `source $SPACK_ROOT/share/spack/setup-env.sh`). Required each time a new shell is opened.

Step 6: Register the compiler.

```
spack compiler find /usr/bin
```

`nvhpc@24.7` and `gcc@8.5.0` are registered into `$WORK/.spack/compilers.yaml`.

Step 7: Register the SEMSWS Spack repo.

```
spack repo add $WORK/semsws/spack-repo
```

Step 8: Create and activate a Spack environment.

```
spack env create semsws-gpu
spack env activate semsws-gpu
```

Using a separate env lets the GPU and CPU builds coexist.

Step 9: Edit `spack.yaml`.

```
spack config edit
```

The editor opens. Paste the `packages:` section of `spack-repo/site-config/es/packages_gpu.yaml` nested under the `spack:` key, and add the specs and concretizer settings. The final form:

```
spack:
  repos:
    - /S/data01/G3506/<USER>/program_test/semsws/spack-repo
  packages:
    # paste the body of packages_gpu.yaml here
    # (externals for nvhpc / openmpi / cuda / cmake,
    # require: '%gcc' for hdf5 / metis / netcdf-c / openblas, etc.)
  specs:
    - semsws@main +cuda cuda_arch=80 precision=single +gpu_aware_mpi
      cxxflags=="-noswitcherror" cflags=="-noswitcherror"
      ldlibs=="-lstdc++fs"
      ^hypre@2.33
      %nvhpc
  concretizer:
    unify: true
  config:
    build_jobs: 4
```

The meaning of each spec flag is in §2.5.8; the rationale for each entry in `packages_gpu.yaml` is in §2.5.8.

Step 10: Resolve dependencies.

```
spack concretize -f
```

Regenerates `spack.lock` from `spack.yaml`. `-f` forces a re-solve.

Step 11: Install.

```
spack install --fail-fast 2>&1 | tee install_gpu.log
```

`--fail-fast` stops immediately when any dependency fails. The first full build takes 50–90 minutes (hypre + the CUDA kernels compiled by `nvcc` account for more than 30 minutes). Running this inside `tmux` is recommended so the progress UI (`[+]` `[-]` `[x]` animations) stays visible.

Step 12: Load and verify.

```
spack load semsws
which semsws
# /S/data01/.../spack/opt/spack/.../semsws-main-.../bin/semsws
semsws --help
```

2.5.4 Earth Simulator (CPU)

The CPU partition. System OpenMPI 4.0.5 + gcc 8.5 + single precision, without NVHPC. Managed as a separate env so it can coexist with the GPU env.

Step 1: Steps 1–3 are shared with the GPU procedure (WORK setup, Spack clone, SEMSWs clone).

Step 2: Copy the CPU load script to home.

```
cp spack-repo/site-config/es/load_cpu.sh ~/load_cpu.sh
chmod +x ~/load_cpu.sh
```

Step 3: Exit the GPU env (if any) and load the CPU environment.

```
spack env deactivate      # only if currently in the GPU env
source ~/load_cpu.sh
```

NVIDIAHPCSDK and OpenMPI/4.0.5 cannot be loaded at the same time, so switching environments requires going through `load_cpu.sh`, which calls `module purge`.

Step 4: Register the compiler and the repo.

```
spack compiler find /usr/bin      # gcc@8.5 is registered
spack repo add $WORK/semsws/spack-repo
```

Skip `spack repo add` if already done in the GPU env.

Step 5: Create the CPU env and edit `spack.yaml`.

```
spack env create semsws-cpu
spack env activate semsws-cpu
spack config edit
```

Paste `spack-repo/site-config/es/packages_cpu.yaml`, and set the specs to:

```
specs:
- semsws@main precision=single ^hypre@2.33 %gcc
```

Main differences from the GPU env (§2.5.3 Step 9):

- Remove the `nvhpc` / `cuda` external declarations; `gcc` is the only compiler.
- `openmpi` becomes an external pointing to `/opt/share/OpenMPI/4.0.5`.
- Drop `+cuda`, `+gpu_aware_mpi`, and `cxxflags=="-noswitcherror"` from the spec, and switch to `%gcc`.
- Keep `^hypre@2.33` for safety (the CMake FindMPI issue can also affect CPU builds).

Step 6: Resolve dependencies and install.

```
spack concretize -f
spack install --fail-fast 2>&1 | tee install_cpu.log
```

2.5.5 Second and subsequent sessions (ES)

```
# Use the GPU build
source ~/load_gpu.sh
spack env activate semsws-gpu
spack load semsws

# Use the CPU build
source ~/load_cpu.sh
spack env activate semsws-cpu
spack load semsws
```

Why three commands are needed:

- `source ~/load_*.sh`: the system modules for GPU and CPU are mutually exclusive (NVIDIAHPCSDK and the system OpenMPI cannot be loaded together), so this is required every time. Spack env activate alone does not switch the module environment.
- `spack env activate semsws-gpu`: activates the corresponding env's `spack.yaml`.
- `spack load semsws`: adds the `bin/lib` of the semsws installation to `PATH/LD_LIBRARY_PATH` of the current shell.

After verifying that semsws is on `PATH` via `which semsws`, submit the job with `mpirun`.

2.5.6 Example ES job script (Spack build)

Examples of submitting a Spack-installed semsws through the ES batch job system (PJM).

GPU env (A100 node, 1 node × 8 GPUs)

```
#!/bin/bash
#PJM -L rscgrp=gpu
#PJM -L node=1
#PJM -L elapse=01:00:00
#PJM -j

source ~/load_gpu.sh
spack env activate semsws-gpu
spack load semsws

# 8 ranks on a single node (each bound to one of the 8 A100s)
mpirun -np 8 -map-by ppr:8:node \
    semsws --config config.yaml
```

CPU env (CPU node, 1 node × 16 ranks)

```
#!/bin/bash
#PJM -L rscgrp=cpu
#PJM -L node=1
#PJM -L elapse=04:00:00
#PJM -j

source ~/load_cpu.sh
spack env activate semsws-cpu
spack load semsws

mpirun -np 16 semsws --config config.yaml
```

Multiple nodes For multi-node runs, change to `#PJM -L node=N` and set `mpirun -np` to the total number of ranks ($N*8$ for GPU, $N*16$ for CPU). `mpirun` automatically picks up PJM environment variables such as `NODE_LIST`.

When spanning nodes, **always pass `-x PATH -x LD_LIBRARY_PATH` to `mpirun`**. `spack load semsws` only exports `PATH` and `LD_LIBRARY_PATH` in the head node's shell; without this flag the remote ranks hit `semsws: command not found` or `cannot open shared object file` for shared libraries. Example:

```
mpirun -np 16 \
  --hostfile ${PBS_NODEFILE} \
  --map-by ppr:8:node:PE=16 \
  --bind-to core \
  -x PATH -x LD_LIBRARY_PATH \
  ./run_with_gpu.sh semsws --config config.yaml
```

Submitting the job

```
pjsub job_gpu.sh          # submit
pjstat                   # queue status
pjcat <jobid> -o          # show stdout
```

2.5.7 Running pytest in the Spack environment

```
spack env activate semsws-gpu
spack load semsws
SEMSWS_PREFIX=$(spack find --paths semsws | \
  awk 'NR>1 && /\// {print $NF; exit}')

pytest -v --build-dir $SEMSWS_PREFIX --np 4 --device cuda \
  $WORK/semsws/test/consistency/
```

- `--build-dir`: prefix under which `pytest` searches for the `semsws` binary. With Spack install it is at `$prefix/bin/semsws`.
- `--np`: number of MPI processes. Each SEMSWS test uses a small mesh, so an excessive `np` (128 or more) leaves empty ranks after METIS partitioning and causes segfaults in routines such as `IsotropicElasticMaterial3D::AttenuationQmu()`. A safe range is `np = 4--16`.
- `--device cuda / cpu`: simulation device selection. Use `cuda` for the GPU env and `cpu` for the CPU env.
- The reason for getting `SEMSWS_PREFIX` via `spack find --paths` is that, unlike `spack location -i`, it does not call `git` internally and therefore does not fail in batch jobs.

2.5.8 Appendix: rationale for the ES Spack configuration

Contents of `load_gpu.sh`. What source `~/load_gpu.sh` does:

- `module purge`: reset to a clean module state. `NVIDIAHPCSDK` and `OpenMPI/4.0.5` cannot be loaded at the same time, so `purge` and `reload` from scratch every session.
- `module load NVIDIAHPCSDK/24.7/nvhpc-hpcx-cuda12`: load NVHPC SDK 24.7. HPC-X OpenMPI (CUDA-aware) comes in as well.

- `module load CUDA/12.0.0`: standalone CUDA 12.0 module. The CUDA 12.5 bundled with NVHPC has `cuda/` and `math_libs/` in separate locations and breaks MFEM's `cusparse.h` detection, so the flat-layout 12.0 is used separately (same as the manual build of ES in the previous section).
- `module load METIS/5.1.0 / HDF5/1.10.5`: auxiliary libraries for the build. In the Spack flow most of these are **rebuilt by Spack** (see `packages_gpu.yaml`).
- `module load Python/3.12.6`: used for pytest, etc.
- `module load Git/2.30.1`: the system git on ES is old and occasionally trips up the Spack 1.x startup check and some fetchers, so a newer git is provided explicitly as a module.
- `export PATH=/usr/bin:$PATH`: under ES's batch sandbox, `/usr/bin` can fall off the front of `PATH` and `/usr/bin/git` becomes invisible; prepend it explicitly.
- `export SPACK_GIT_COMMAND=/usr/bin/git / GIT_PYTHON_GIT_EXECUTABLE=/usr/bin/git`: the "spack requires 'git'" check at Spack 1.x startup looks for git in the sanitized `PATH`, so tell it explicitly.
- `export SPACK_USER_CONFIG_PATH=$WORK/.spack / SPACK_USER_CACHE_PATH=$WORK/.spack-cache`: keep Spack config and the download cache under work (not home), centralizing the multi-env setup in one place.
- `source $SPACK_ROOT/share/spack/setup-env.sh`: injects Spack's shell functions (`spack` command, `spack env activate`, `spack load`) into the current shell. Without this, subsequent `spack` invocations fail with `command not found`.

Meaning of each spec flag.

- `+cuda cuda_arch=80`: target `sm_80` for A100. Propagates automatically to `hypr` and `mfem`.
- `precision=single`: propagates to `mfem` and `hypr`. Standard for FWI.
- `+gpu_aware_mpi`: HPC-X OpenMPI is already built CUDA-aware, so this can be enabled. Also propagates to `hypr`'s `+gpu-aware-mpi`.
- `cxxflags=="-noswitcherror" cflags=="-noswitcherror"`: `yaml-cpp` passes the gcc-only flags `-Weffc++` and `-pedantic-errors` to `nvc++`, which would error out. The flag tells `nvc++` to "ignore unknown flags" and is propagated across all packages (`==` is sticky propagation).
- `ldlibs=="-lstdc++fs"`: gcc 8.5 on rhel8 separates `std::filesystem` into the dedicated archive `libstdc++fs`. Adding it explicitly lets `adios2` (which assumes gcc 9+, where `filesystem` is merged into `libstdc++`) link successfully.
- `^hypr@2.33`: `hypr` 3.x suffers a chain of issues on ES gcc 8.5 (`cstddef` + extern "C" collision, an incompatible CMake FindMPI, additional dependencies through UMPIRE, etc.), so the 2.33 series is pinned explicitly. The shadow recipe (`spack-repo/packages/hypr/package.py`) disables `cuSOLVER`, which also bypasses the single-precision FSAI bug inside `hypr`.
- `%nvhpc`: build SEMSWs itself with `nvc++`. Independent of gcc-forced packages such as `openblas` and `netcdf-c`.

Entries in `packages_gpu.yaml`.

- `nvhpc`: prefix set to `/opt/share/NVIDIAHPCSDK/24.7`. **Do not include the compilers/subdirectory**. The Spack `nvhpc` package internally appends `<prefix>/Linux_x86_64/<version>/comp` so setting the prefix to `.../compilers` duplicates that path and `libblas` cannot be found.
- `openmpi`: HPC-X 4.1.7 bundled with NVHPC. Register as an external only, declaring `+cuda fabrics=ucx`.
- `cuda`: points to `/opt/share/CUDA/12.0.0`. The CUDA 12.5 bundled with NVHPC has a split layout where `cusparse.h` lives under `math_libs`, and MFEM's `find_package(CUDAToolkit)` fails.
- `hdf5` / `metis` / `netcdf-c` / `netcdf-cxx4` / `netcdf-fortran` / `openblas` are all forced to `require: '%gcc'` so that Spack builds them. The reasons are that `nvc`'s autotools probes choke on `-m32` or `-Wno-error`, and that the system modules are serial-only and incompatible with `+mpi`. Linking gcc-built C libraries into an `nvc++`-built mfem is ABI-safe because `libstdc++` and `libc` are shared.
- `hypr`: `variants: +shared`. `hypr`'s CUDA backend pulls in `libcudart/libcusparse` symbols, which are propagated to consumers (mfem, SEMSWs) automatically through the `DT_NEEDED` list of the shared object (`.so`). A static archive has no `DT_NEEDED`, so each consumer would have to add `-lcudart` etc. explicitly to its link line.

2.6 Verifying the Build

When the build succeeds, the executables are produced under `build/src/`.

Table 2.5: Generated executables

Executable	Description
<code>semsws</code>	Simulation main binary
<code>partition_mesh</code>	Pre-partition a mesh
<code>refine_mesh</code>	Refine a mesh
<code>evaluate_mesh</code>	Evaluate mesh quality
<code>combine_mesh</code>	Combine a partitioned mesh (for visualization)
<code>bp2gf</code>	Convert ADIOS2 BP to GLVis format

To verify that the build completed correctly:

```
./build/src/semsws --help
```

The build can also be validated via the test suite. Running the tests requires Python 3.9 or later, `pytest`, and `numpy`.

```
# Run all tests (default: CPU, serial)
pytest test/ --build-dir ./build -v

# MPI parallel run with GPU device
pytest test/ --build-dir ./build --np 4 --device cuda -v

# Waveform verification tests only
pytest test/waveform/ --build-dir ./build -v

# Output-format consistency tests only
pytest test/consistency/ --build-dir ./build -v
```

Two kinds of tests are provided.

Waveform verification tests (test/waveform/) Compare the simulation result to a reference waveform and verify that the normalized L2 norm is within a threshold. Cases whose name contains **analytic** are compared against an analytical solution; all other cases are compared against SPECSEM. The following cases are included:

Table 2.6: Waveform verification test cases

Dimension	Physical model
2D	acoustic, elastic, acoustic_visco, elastic_visco
3D	acoustic_analytic, elastic, elastic_analytic, elastic_visco

Running with the `--keep-results` option preserves the simulation outputs under each test directory's `results/` folder (e.g., `test/waveform/3D/elastic/`), so they can be compared file by file against the reference waveforms in the `ref/` folder of the same directory:

```
pytest test/waveform/ --build-dir ./build --keep-results -v
```

Output-format consistency tests (test/consistency/) Verify that, across receiver output formats (ASCII, HDF5, SU), the recorded waveforms agree numerically.

2.7 Installing GLVis (Recommended)

GLVis is a lightweight visualization tool published by the MFEM project². It can directly read the MFEM-format mesh and grid-function files that SEMSWS writes to `results/wavefield/`, and display the wavefield frame by frame either while a simulation is running or after it finishes. GLVis is not required for SEMSWS itself to run, but installing it makes the Quick Start tutorial in Chapter 3 much easier to follow (one can verify the simulation interactively with a single command before writing a plotting script).

2.7.1 Building

GLVis can be built using the MFEM installed in the previous section as-is.

```
cd ~/program_ubuntu
git clone https://github.com/GLVis/glvis.git
cd glvis
make -j MFEM_DIR=../mfem
```

The executable is produced directly under the checkout as `glvis`. Add it to `PATH`, or otherwise make it callable by its full path.

The system packages `SDL`, `OpenGL`, `fontconfig`, and `freetype` are required. On Debian/Ubuntu:

```
sudo apt install libsdl2-dev libglew-dev libfontconfig-dev \
    libfreetype-dev
```

²<https://glvis.org/>

2.7.2 Basic usage

Specify either a mesh alone, or a mesh together with a grid function produced by SEMSWs, to launch GLVis:

```
# Mesh only
glvis -m mesh.mesh

# Mesh + a single-frame wavefield snapshot
glvis -m results/wavefield/mesh.mesh -g results/wavefield/sol_000500.gf

# Partitioned (MPI-parallel) mesh - N is the number of output ranks
glvis -np 2 -m results/wavefield/mesh -g results/wavefield/sol_000500
```

In the partitioned mesh case, GLVis automatically reads `mesh.000000`, `mesh.000001`, ...together with the corresponding `sol_000500.000000` ... files and merges them into a single window.

Refer to the GLVis README³ for additional launch options and window keybindings.

³<https://github.com/GLVis/glvis>

Chapter 3

Quick Start

This chapter walks through the complete workflow—from writing a configuration file to running a simulation and visualizing the results—using a 2D acoustic wave simulation as an example. The example used here is included in `examples/visualization/2D/acoustic/`.

3.1 Configuration File

A SEMSWS simulation is fully specified by a single YAML file (`config.yaml`) that contains all parameters. This section walks through the configuration file of this example section by section.

3.1.1 Basic simulation settings

```
simulation:
  dimension: 2
  order: 4

  time:
    steps: 10000
    dt: 0.0001
```

`dimension` specifies the spatial dimension (2 or 3), and `order` specifies the polynomial order (the order of the spectral element method). The `time` section sets the number of time steps and the time-step width Δt . In this example, $\Delta t = 0.0001$ s and 10000 steps give a 1 s simulation.

3.1.2 Mesh

```
mesh:
  type: internal
  origin: [0.0, 0.0]
  size: [1000.0, 500.0]
  elements: [50, 25]
  save: true
  max_freq: 75.0
  ppw: 5.0
```

`type: internal` tells SEMSWS to generate a regular/structured mesh automatically inside the solver. `size` specifies the domain size (m) and `elements` the number of elements. This example partitions a 1000 m $\times$ 500 m domain into 50 $\times$ 25 elements. Setting `save` writes a mesh for GLVis visualization into the results directory. `max_freq` and `ppw` are parameters used to assess mesh quality: they denote the maximum frequency and the number of points per

wavelength (points per wavelength). They are printed in `summary.txt` and consumed by the `evaluate_mesh` tool described later.

To use an external mesh (GMSH, CUBIT/ExodusII, etc.), set `type: external` and provide the file path in `file:`.

3.1.3 Material

```
material:
  type: isotropic_acoustic
  format: constant
  vp: 1500.0
  rho: 1000.0
  attenuation:
    enabled: false
```

This defines a homogeneous acoustic medium. `type` selects the physical model (`isotropic_acoustic`), and `format: constant` declares that the model is homogeneous. The P-wave velocity is 1500 m/s and the density is 1000 kg/m³.

3.1.4 Boundary conditions

```
boundary:
  absorbing:
    type: cerjan
    sides: []
    thickness: 100.0
    alpha: 0.4

  dirichlet:
    attributes: [top, right, left, bottom]
```

`dirichlet` applies Dirichlet (fixed) boundary conditions on all sides. The absorbing boundary (Cerjan sponge layer) is disabled by `sides: []`. To enable the absorbing boundary, list the sides on which to apply it, e.g., `sides: [top, right, left, bottom]`. For marine reflection or refraction seismic surveys, the **sea surface must be set as Dirichlet** (`dirichlet: [top]`).

3.1.5 Source

```
sources:
  list:
    - id: 1
      name: "Source 1"
      type: pressure
      location: [500.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
```

A pressure point source with a 30 Hz Ricker wavelet is placed near the center of the domain (500 m, 400 m). `delay` is the time of the wavelet peak (s).

3.1.6 Receivers

```
receivers:
  output:
    format: ascii
    type: [PS]

  list:
    - name: "R001"
      location: [400.0, 300.0]
    - name: "R002"
      location: [500.0, 300.0]
    - name: "R003"
      location: [600.0, 300.0]
```

Three receivers are placed, recording the pressure scalar field (PS). The output format is `ascii`: a text file with time in the first column and amplitude in the second. Other options for `format` are `hdf5` and `su`; `su` is the Seismic Unix format.

3.1.7 Wavefield output

```
simulation:
  output:
    wavefield:
      enabled: true
      interval: 1000
      fields: ["PS"]
      formats:
        - type: "glvis"
        - type: "paraview"
          refinement: 4
          data_format: "binary32"
        - type: "gmt"
          resolution: [100, 50]
```

Wavefield snapshots are output every 1000 steps. Three output formats can be specified simultaneously:

- **GLVis** — MFEM native format. Interactively visualized with the `glvis` command.
- **ParaView** — VTU format. Suited for 3D visualization in ParaView.
- **GMT** — Text grid format (`x y value`). `resolution` specifies the grid resolution.

3.1.8 Device settings

```
device:
  type: cpu
```

Choose from `cpu`, `cuda`, or `hip`. Even on a GPU build, specifying `cpu` runs on the CPU (but this is very slow; for CPU runs, use a CPU build).

3.2 Running the Simulation

Run the following command in the directory containing the configuration file:

```
cd examples/visualization/2D/acoustic

# Run with 4 processes
mpirun -np 4 ../../../../build/src/semsws \
  --config config.yaml
```

You can also use the `run.sh` script included in the same directory:

```
bash run.sh 4    # argument is the number of MPI processes
```

When execution completes, the following are produced under the `results/` directory:

Table 3.1: List of output files

Path	Description
<code>summary.txt</code>	Simulation information (material, mesh, performance, etc.)
<code>R001_0001.p</code> , etc.	Receiver waveforms (ASCII text)
<code>wavefield/gmt/</code>	GMT snapshots
<code>wavefield/paraview/</code>	ParaView VTU files
<code>wavefield/glvis/</code>	GLVis binary files
<code>material/</code>	Material property output
<code>mesh.mesh</code>	Mesh that was used

3.3 Checking the Results

3.3.1 Summary

`results/summary.txt` records a summary of the simulation. Example output from this case:

```
Discretization:
  Order: 4
  Elements: 1250
  DOFs: 20301

Time stepping:
  dt: 0.0001 s
  nt: 10000
  Simulation time: 1 s

Numerical stability:
  h_min: 20.00 m, h_max: 20.00 m
  CFL dt_max: 7.3535e-04 s (dt/dt_max = 0.14)
  PPW: 12.5 (f=30.0 Hz)

Performance:
  Setup time: 0.18 s
  Run time: 2.27 s
  Total time: 2.44 s
  DOFs/sec: 8.96e+07
```

Verify in particular that `CFL dt_max` is larger than the configured Δt . If $\Delta t > \Delta t_{\max}$, the computation is unstable and diverges. PPW (Points Per Wavelength) is the number of grid points per wavelength, with 5 or more as a common rule of thumb.

3.3.2 Wavefield snapshots

The bundled Python script visualizes the wavefield:

```
python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results
```

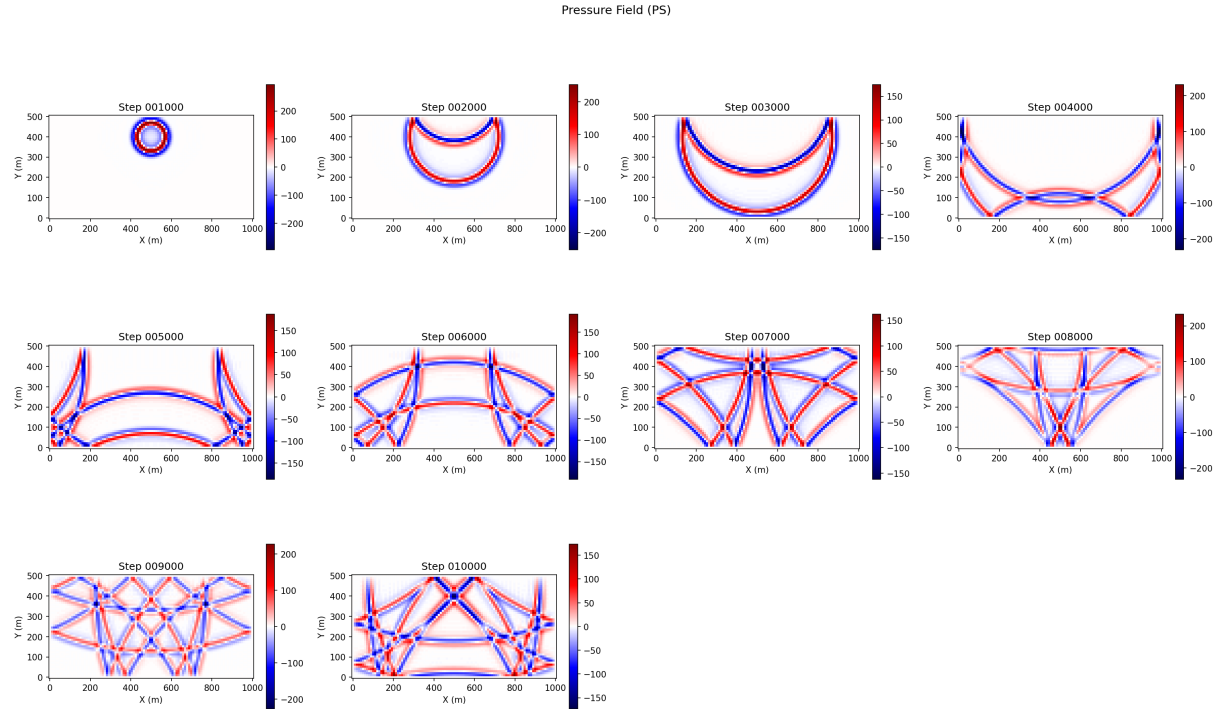


Figure 3.1: Pressure field snapshots (every 1000 steps).

Because this example imposes Dirichlet boundary conditions on all sides, the wave is reflected at the boundary surfaces. Actual seismic wave simulations use absorbing boundary conditions to suppress such artificial reflections.

3.3.3 Receiver waveforms

The receiver waveform files are 2-column ASCII text. The first column is the time (seconds) and the second column is the amplitude (pressure). They can be readily plotted with gnuplot, Python, or similar tools.

Chapter 4

Input File Reference

A SEMSW simulation specifies all of its parameters in a single YAML file (`config.yaml`). This chapter describes every option in the configuration file exhaustively. For basic usage, refer to Chapter 3.

4.1 YAML Format Basics

The configuration file is written in YAML (YAML Ain't Markup Language). The basic syntax rules are:

- Indentation with spaces expresses the hierarchical structure.
- Keys and values use the `key: value` form.
- Lists are expressed with `-`.
- Anything after `#` is treated as a comment.
- Strings may be unquoted; surround them with `"..."` when they contain special characters.

```
# Scalar values
name: "My Simulation"
order: 4
dt: 0.0001

# Nested structure
simulation:
  dimension: 2
  time:
    steps: 10000
    dt: 0.0001

# List
sides: [bottom, right, top, left]

# List (multi-line form)
list:
  - name: "R001"
    location: [400.0, 300.0]
  - name: "R002"
    location: [500.0, 300.0]
```

4.2 Strict-Key Validation

The YAML loader checks each block of the configuration file against a known set of keys. If an unknown key (a typo, an old key that has been renamed, a sub-block copy-pasted from an outdated configuration, etc.) appears anywhere, it is not silently ignored; instead, execution aborts with a runtime error of the form:

```
Error validating config: Unknown key `<bad_key>` in <section>
  (allowed: <list of valid keys for that block>)
```

The same mechanism rejects removed legacy keys with a dedicated message:

- `boundary.free_surface` has been removed. Free surfaces in elastic regions are a natural boundary condition, so the entire block can be deleted. Free surfaces in acoustic regions (pressure = 0) should instead be declared via `boundary.dirichlet.attributes`.
- `simulation.output.wavefield.format` and `receivers.output.format` (singular) have been replaced by `formats`: (a list of mapping elements, plural). See §4.5.5 and §4.10.1.
- `component`: <int> (singular) inside an output-format element has been replaced with `components`: [<int>, ...].
- `simulation.checkpoint_storage`: `host_memory` has been renamed to `memory`.
- `material.format`: `ascii` has been renamed to `grid`.

When porting an old configuration file, apply the renames above and remove any keys not listed in this chapter.

4.3 Overall Structure of the Configuration File

The overall structure of the configuration file is:

```
name: "... "           # simulation name (optional)
description: "... "     # description (optional)
simulation:             # basic simulation settings and outputs
mesh:                  # mesh
material:              # material
boundary:              # boundary conditions
sources:               # sources
receivers:             # receivers
device:                # execution device
```

4.4 Root-Level Parameters

Key	Type	Required	Default	Description
name	string	optional	"Seismic wave simulation"	Simulation name. Recorded in summary.txt
description	string	optional	"No description provided."	Free-form description

4.5 simulation section

4.5.1 Basic parameters

Key	Type	Required	Values	Description
<code>dimension</code>	int	required	2, 3	Spatial dimension
<code>order</code>	int	required	≥ 1	Polynomial order (number of GLL points = $N + 1$)

On the GPU implementation, `order` has an upper limit imposed by the shared-memory constraint.

4.5.2 Time integration (time)

Key	Type	Required	Default	Description
<code>time.dt</code>	real	required	—	Time-step width (s)
<code>time.steps</code>	int	required	—	Number of time steps
<code>time.cfl_factor</code>	real	required	—	CFL safety factor (0–0.7)
<code>time.t0</code>	real	optional	0.0	Simulation start time (s)

Δt must satisfy the CFL condition ($\Delta t < \Delta t_{\max}$). `cfl_factor` is used in mesh quality evaluation. The total simulation time is determined by `steps` $\times$ Δt .

4.5.3 Simulation mode

Key	Type	Required	Default	Description
<code>mode</code>	string	optional	<code>forward</code>	<code>forward</code> , <code>inversion</code> , <code>misfit_only</code>
<code>misfit_type</code>	string	optional	<code>l2_waveform</code>	<code>l2_waveform</code> , <code>normalized_correlation</code>
<code>num_checkpoints</code>	int	optional	10	Number of Revolve checkpoints
<code>checkpoint_storage</code>	string	optional	<code>"memory"</code>	<code>"disk"</code> , <code>"memory"</code>
<code>checkpoint_device</code>	string	optional	<code>"auto"</code>	<code>"host"</code> , <code>"device"</code> , <code>"auto"</code>
<code>checkpoint_dir</code>	string	conditional	<code>"./checkpoints/"</code>	Required when <code>disk</code> (not supported yet)
<code>kernel_output_dir</code>	string	optional	<code>"./kernels/"</code>	Destination for sensitivity kernels

`forward` is the standard forward simulation. `inversion` and `misfit_only` are for Full Waveform Inversion (FWI): they compute the gradient using a checkpointing algorithm + adjoint, and compute waveform misfits, respectively. They are currently supported only for 2D acoustic simulations. The other keys are also for FWI and have not yet been fully debugged.

4.5.4 Output settings (output)

Key	Type	Required	Default	Description
<code>output.directory</code>	string	optional	<code>"./"</code>	Root output directory
<code>output.summary_file</code>	string	optional	<code>""</code>	Summary file name (empty string disables)
<code>output.log_interval</code>	int	optional	100	Log output interval (in steps)

All output files are written under the directory specified by `output.directory`. `output.summary_file` writes a summary of the run; it is convenient to set this to e.g. `summary.txt`. An empty string disables it. `output.log_interval` controls how often (in steps) the live progress is printed. Each progress report involves an MPI reduction, so set it to a reasonable value. In particular, on GPU runs each report induces host/device communication, so be careful.

4.5.5 Wavefield output (output.wavefield)

Key	Type	Required	Default	Description
<code>enabled</code>	bool	optional	false	Enable wavefield snapshot output
<code>interval</code>	int	conditional	—	Output interval (in steps)
<code>fields</code>	string array	optional	<code>["DISP"]</code>	Fields to output
<code>formats</code>	array	conditional	—	List of output formats

Values allowed in `fields`: `DISP` (displacement), `VEL` (velocity), `ACC` (acceleration), `PS` (pressure scalar, only for acoustic wave simulations).

Output formats Each element of the `formats` array takes the following options:

Key	Applies to	Default	Description
<code>type</code>	all	—	<code>glvis</code> , <code>paraview</code> , <code>gmt</code>
<code>refinement</code>	paraview	1	Subdivision level
<code>data_format</code>	paraview	<code>binary32</code>	<code>ascii</code> , <code>binary</code> (64-bit), <code>binary32</code> (32-bit)
<code>compression</code>	paraview	−1	zlib compression level (0–9, −1=default; likely requires MFEM built with zlib, untested)
<code>resolution</code>	gmt	—	Number of output grid points (resolution) [<code>nx</code> , <code>ny</code>]
<code>components</code>	gmt	[−1]	−1=magnitude, 0=x, 1=y, 2=z (multiple values allowed, e.g. [−1, 0, 1, 2])
<code>cross_sections</code>	gmt (3D)	—	Cross-section locations

For 3D GMT output, specify cross-section locations with `cross_sections`:

```
cross_sections:
  yz: [2500.0]      # yz cross-section at x=2500 m
  xz: [2500.0]      # xz cross-section at y=2500 m
  xy: [1000.0]      # xy cross-section at z=1000 m
```

Per-side override (for fluid–solid coupling). When `material.type: coupled` is used, the fluid and solid sides live on separate `ParSubMeshes`, and the wavefield output files are also written separately per side. Adding a `fluid:` or `solid:` sub-block within the `wavefield:` block lets you override `enabled` / `interval` / `fields` / `formats` / `components` for that side only. Unspecified keys inherit from the parent `wavefield:` settings. Setting `enabled: false` on one side stops output for that side while keeping the other side intact.

```

wavefield:
  enabled: true
  interval: 500
  formats:
    - type: glvis
  fluid:
    fields: [PS]           # pressure only on the fluid side
  solid:
    fields: [DISP, VEL]    # displacement and velocity on the solid side

```

4.5.6 Material output (output.material)

Key	Type	Required	Default	Description
<code>enabled</code>	bool	optional	false	Enable material output
<code>fields</code>	string array	optional	["vp"]	Fields to output
<code>formats</code>	array	conditional	—	Formats (same as for wavefield)

Values allowed in `fields`: `vp`, `vs`, `rho`, `qkappa`, `qmu`. The material output is written only once at the start of the simulation.

For simulations with `material.type: coupled`, just like the wavefield case, `fluid:` and `solid:` sub-blocks may be placed under `output.material` to override `enabled` / `fields` / `formats` per side (for example, to output `vs` only on the solid side).

4.5.7 Example settings for the simulation section

```

simulation:
  dimension: 3
  order: 4

  time:
    steps: 20000
    dt: 0.00005
    cfl_factor: 0.3

  output:
    directory: "./results"
    summary_file: "summary.txt"
    log_interval: 500

  wavefield:
    enabled: true
    interval: 1000
    fields: ["DISP"]
    formats:
      - type: "paraview"
        refinement: 2

```

```

    data_format: "binary32"
  - type: "gmt"
    resolution: [200, 200]
    components: [-1, 0, 1, 2]
    cross_sections:
      xy: [0.0]
      xz: [2500.0]

  material:
    enabled: true
    fields: ["vp", "vs", "rho"]
    formats:
      - type: "paraview"
        refinement: 2

```

4.6 mesh section

4.6.1 Common parameters

Key	Type	Required	Default	Description
type	string	required	—	internal, external, partitioned
max_freq	real	optional	—	Maximum frequency (Hz) used in mesh quality evaluation
ppw	real	optional	—	Points Per Wavelength
save	bool	optional	false	Save the mesh to the results directory

4.6.2 Internal mesh (type: internal)

SEMSWS generates a structured grid mesh internally.

Key	Type	Required	Default	Description
origin	real array	required	—	Origin coordinates [x,y] or [x,y,z] (m)
size	real array	required	—	Domain size [Lx,Ly] or [Lx,Ly,Lz] (m)
elements	int array	required	—	Number of elements [Nx,Ny] or [Nx,Ny,Nz]
partition	string	optional	metis	metis or cartesian
partition_grid	int array	conditional	—	Partition counts [nx,ny,nz] when using cartesian
attr_y_threshold	real	optional	—	If specified, assigns attribute = 1 to elements whose y coordinate (2D) / z coordinate (3D) is below this value, and attribute = 2 to those above. Convenient for setting up simple fluid–solid coupling meshes.

`partition: cartesian` performs a structured partitioning that does not require METIS. With `partition_grid`, e.g. [4, 4, 2] in 3D splits the domain into 4, 4, and 2 along the x, y, and z axes respectively. The parallelism of the run command (e.g., `--np 4`) must equal the product (32 in this case).

4.6.3 External mesh (type: external)

Key	Type	Required	Default	Description
<code>file</code>	string	required	—	Mesh file path
<code>format</code>	string	optional	auto-detected	<code>gms</code> , <code>mfem</code> , <code>exodus</code> , <code>vtk</code>
<code>partition</code>	string	optional	<code>metis</code>	MPI partitioning method

To use the ExodusII format (CUBIT), MFEM must be built with `MFEM_USE_NETCDF=ON`.

4.6.4 Pre-partitioned mesh (type: partitioned)

Reads an MFEM-format mesh that has been pre-partitioned by the `partition_mesh` tool. Each MPI rank reads only its own partition file. Useful for large meshes or when the same mesh is reused across many simulations.

Key	Type	Required	Default	Description
<code>partitioned.directory</code>	string	required	—	Directory of partition files
<code>partitioned.nparts</code>	int	required	—	Number of partitions (must match the MPI process count)

The file naming convention is `mesh.mesh.000000`, `mesh.mesh.000001`, ...

4.6.5 Example settings for the mesh section

Internal mesh:

```
mesh:
  type: internal
  origin: [0.0, 0.0, 0.0]
  size: [5000.0, 5000.0, 3000.0]
  elements: [50, 50, 30]
  max_freq: 20.0
  ppw: 5.0
  save: true
  partition: cartesian
  partition_grid: [4, 4, 2]
```

External mesh:

```
mesh:
  type: external
  file: "./meshes/model.exo"
  max_freq: 15.0
  ppw: 5.0
```

Pre-partitioned mesh:

```
mesh:
  type: partitioned
  partitioned:
    directory: "./meshes/partitioned"
    nparts: 32
  max_freq: 15.0
  ppw: 5.0
```

4.7 material section

4.7.1 Material types and formats

type value	Description
<code>isotropic_acoustic</code>	Isotropic acoustic medium (vp, rho)
<code>isotropic_elastic</code>	Isotropic elastic medium (vp, vs, rho)
<code>coupled</code>	Fluid–solid coupled medium. The fluid side is <code>isotropic_acoustic</code> and the solid side is <code>isotropic_elastic</code> . The <code>fluid:</code> and <code>solid:</code> sub-blocks declare each side independently. See §4.7.8.

format value	Description
<code>constant</code>	Homogeneous properties specified directly in YAML
<code>grid</code>	ASCII file of a structured grid
<code>by_attribute</code>	Text file giving properties per mesh attribute
<code>by_attribute_mixed</code>	YAML file mixing constant/grid specifications per attribute
<code>adios2</code>	ADIOS2 BP file (for FWI)

When `type: coupled` is used, `format` is not specified at the top level. The nested `fluid:` and `solid:` sub-blocks each carry their own `format`.

4.7.2 format: constant

Specify material properties directly in YAML.

Key	Type	elastic	acoustic
<code>vp</code>	real	required	required
<code>vs</code>	real	required	—
<code>rho</code>	real	required	required

Units are m/s, m/s, and kg/m³ respectively.

```
material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
```

4.7.3 format: grid

Read heterogeneous material values on a structured grid from an ASCII file. Specify the path with `file`.

File format:

```
Nx Ny [Nz]           # number of grid points
dx dy [dz]           # grid spacing (m)
x0 y0 [z0]           # origin (m)
vp vs rho [Qkappa Qmu] # property values at each grid point
vp vs rho [Qkappa Qmu] # (Nx*Ny[*Nz] rows)
...
```

In the elastic case there are 5 columns (vp, vs, rho, Qkappa, Qmu); in the acoustic case there are 3 columns (vp, rho, Qkappa). The Q-value columns are only required if attenuation is enabled.

Data are stored in lexicographic order (x varies fastest). The correspondence between index and coordinates is:

2D Row i corresponds to grid point (i_x, i_y) .

$$\begin{aligned} i &= i_y \times N_x + i_x \\ x &= x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy \end{aligned}$$

That is, scan N_x points along x and then move one step along y .

3D Row i corresponds to grid point (i_x, i_y, i_z) .

$$\begin{aligned} i &= i_z \times N_x \times N_y + i_y \times N_x + i_x \\ x &= x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy, \quad z = z_0 + i_z \times dz \end{aligned}$$

That is, scan in the order $x \rightarrow y \rightarrow z$. Points outside the grid are clamped to the nearest boundary value.

The origin (x_0, y_0) or (x_0, y_0, z_0) is the minimum-coordinate point of the grid, and is usually set to match the mesh `origin`. **In the semswws coordinate system, the y axis (2D) / z axis (3D) is positive upward, so the origin is located at the deepest point of the domain. Consequently, the first row of the file starts from the material values at the deepest depth.**

4.7.4 format: by_attribute

A text file that defines material values per mesh element attribute. Used when an external mesh (e.g., CUBIT) has attributes attached.

#	attr	vp	vs	rho	[Qkappa	Qmu]
1		6000.0	3500.0	2700.0	100.0	100.0
2		3000.0	2000.0	2200.0	50.0	50.0

4.7.5 format: by_attribute_mixed

An external YAML file that mixes constant (homogeneous) and grid (file) specifications per attribute.

```
attributes:
  1:
    mode: constant
    vp: 5500.0
    vs: 3175.0
    rho: 2800.0
  2:
    mode: grid
    file: material_layer2.dat
```

4.7.6 format: adios2

Reads material data on GLL points produced by an FWI iteration in the ADIOS2 BP format. A BP file produced by the `semsws_export_model` tool (equivalent of §??) can be fed in directly.

Key	Type	Required	Description
<code>vp_file</code>	string	required	Path to the BP file for V_p
<code>vs_file</code>	string	conditional	Path to the BP file for V_s (required when <code>type: isotropic_elastic</code>)
<code>rho_file</code>	string	required	Path to the BP file for ρ
<code>qkappa_file</code>	string	optional	Path to the BP file for Q_κ (needed for viscoacoustic/viscoelastic)
<code>qmu_file</code>	string	optional	Path to the BP file for Q_μ (needed for viscoelastic)

The `adios2` format is currently not supported inside the `fluid:` / `solid:` sub-blocks of `type: coupled`.

4.7.7 Attenuation (`attenuation`)

Configure attenuation via a generalized Zener model (Standard Linear Solid: SLS).

Key	Type	Required	Default	Description
<code>attenuation.enabled</code>	bool	optional	false	Enable attenuation
<code>attenuation.f0</code>	real	conditional	1.0	Reference frequency (Hz). Required when <code>enabled=true</code>
<code>attenuation.n_units</code>	int	conditional	3	Number of SLS relaxation mechanisms. Required when <code>enabled=true</code>
<code>attenuation.Qkappa</code>	real	conditional	—	Q for bulk modulus (required when <code>format: constant</code>)
<code>attenuation.Qmu</code>	real	conditional	—	Q for shear modulus (required when <code>format: constant</code> and <code>elastic</code>)
<code>attenuation.Qkappa_file</code>	string	optional	—	Q_κ grid file (when <code>format: grid/by_attribute</code> and Q is given via a file)
<code>attenuation.Qmu_file</code>	string	optional	—	Q_μ grid file (likewise for <code>elastic</code>)

The Q -value fitting band is $[0.1f_0, 10f_0]$. With `format: grid` or `by_attribute`, the Q -value column in the file is normally used as-is, so the YAML keys above are not needed. Use `Qkappa_file`/`Qmu_file` to explicitly load Q from a separate file.

4.7.8 type: coupled (fluid–solid coupling)

For simulations where a fluid (acoustic) region and a solid (elastic) region share an interface, specify `material.type: coupled`. In that case the top-level `material:` block has two sub-blocks `fluid:` and `solid:`, each declaring its own `type`, `format`, and material parameters (they are independent; for example, you may have the fluid as `constant` and the solid as `grid`).

Key	Type	Required/Optional	Description
<code>attribute</code>	<code>int</code>	required	The parent mesh element attribute that identifies this sub-region. Must differ between <code>fluid</code> and <code>solid</code> .
<code>type</code>	<code>enum</code>	required	<code>isotropic_acoustic</code> for <code>fluid</code> , <code>isotropic_elastic</code> for <code>solid</code> . <code>coupled</code> cannot be nested.
<code>format</code>	<code>enum</code>	required	Same set as for regular materials: <code>constant</code> , <code>grid</code> , <code>by_attribute</code> , <code>by_attribute_mixed</code> .
<code>vp</code> , <code>vs</code> , <code>rho</code>	<code>real</code>	conditional	Required for <code>format: constant</code> (<code>vs</code> also required on the elastic side).
<code>file</code>	<code>path</code>	conditional	Required for <code>format: grid / by_attribute / by_attribute_mixed</code> .
<code>attenuation</code>	<code>block</code>	optional	If an <code>attenuation:</code> sub-block is provided, it uses the same key set as in non-coupled materials (§4.7.7). Enabling it on the <code>fluid:</code> side gives viscoacoustic behavior; enabling it on the <code>solid:</code> side gives viscoelastic behavior.

The boundary attribute of the fluid–solid interface is detected automatically at runtime by `FluidSolidInterface::Setup` (it picks up boundary attributes that exist on only one of the two `ParSubMeshes` and are not present on the original outer boundary of the parent mesh). No explicit declaration on the YAML side is required.

Allowed format values inside coupled sub-blocks. The format values usable inside the `fluid:` / `solid:` sub-blocks are limited to four: `constant`, `grid`, `by_attribute`, `by_attribute_mixed`. `adios2` and `hdf5` are not currently supported inside coupled sub-blocks.

```
# 2D fluid-solid coupling. Both sides are homogeneous,
# and attenuation (Q_kappa = Q_mu = 40) is enabled in each region.
material:
  type: coupled

  fluid:
    attribute: 1
    type: isotropic_acoustic
    format: constant
    vp: 1500.0
    rho: 1000.0
    attenuation:
      enabled: true
      f0: 5
      n_units: 3
      Qkappa: 40

  solid:
    attribute: 2
    type: isotropic_elastic
    format: constant
    vp: 3000.0
    vs: 1732.0
    rho: 2500.0
    attenuation:
      enabled: true
```

```
f0: 5
n_units: 3
Qkappa: 40
Qmu: 40
```

Requirements on the mesh. The parent mesh must tag all fluid elements with the `fluid.attribute` value and all solid elements with the `solid.attribute` value (the physical group IDs produced by Cubit/GMSH can usually be used directly). In addition, the fluid and solid elements must be topologically connected at the interface. Running only `subtract` in Cubit and skipping `merge vertex` / `merge curve` leaves duplicate vertices and edges, so the elements are not connected, and SEMSWS aborts at runtime with a message indicating the cause.

4.7.9 Model export

Key	Type	Required	Default	Description
<code>export_model</code>	bool	optional	false	Export the model in ADIOS2 BP format
<code>export_dir</code>	string	optional	<code>"./model/"</code>	Export destination directory

4.7.10 Example settings for the material section

Homogeneous elastic medium (with attenuation):

```
material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
  attenuation:
    enabled: true
    f0: 2.0
    n_units: 3
    Qkappa: 100.0
    Qmu: 50.0
```

Definition per attribute (with an external mesh):

```
material:
  type: isotropic_elastic
  format: by_attribute
  file: "materials.txt"
  attenuation:
    enabled: true
    f0: 2.0
    n_units: 3
```

Heterogeneous acoustic medium (grid file):

```
material:
  type: isotropic_acoustic
  format: grid
  file: "velocity_model.dat"
```

4.8 boundary section

4.8.1 Absorbing boundary conditions (absorbing)

Configures the Cerjan sponge-layer absorbing boundary condition.

Key	Type	Required	Default	Description
type	string	required	—	Currently only cerjan
sides	string array	required	—	Sides on which to apply (empty array [] disables)
thickness	real	required	—	Thickness of the sponge layer (m)
alpha	real	required	—	Damping coefficient (4–9)

Side names allowed in **sides**:

Dimension	Allowed sides
2D	bottom , right , top , left
3D	bottom , front , right , back , left , top

Correspondence between side names and boundary attribute IDs. SEMSWs maps each side name to a fixed 1-based MFEM boundary attribute ID. This ID is shared between **sides** and **dirichlet.attributes**. *When creating a mesh with an external mesh generator such as Cubit or GMSH, this ID must be stamped on the corresponding boundary face directly.* A mismatched ID causes the side name in the YAML to refer to the wrong physical boundary.

Table 4.1: Correspondence between side names and the MFEM boundary attribute IDs used by SEMSWs. These are the IDs produced by `Mesh::MakeCartesian2D` and `MakeCartesian3D`, and the same convention must be followed when tagging external meshes.

Side name	2D attribute ID	3D attribute ID
bottom	1	1 (<i>z</i> min)
right	2	3 (<i>x</i> max)
top	3	6 (<i>z</i> max)
left	4	5 (<i>x</i> min)
front	—	2 (<i>y</i> min)
back	—	4 (<i>y</i> max)

dirichlet.attributes accepts a mixture of side names and integer attribute IDs. A side name is merely an alias for the corresponding integer ID in the table above.

4.8.2 Dirichlet boundary conditions (dirichlet)

Key	Type	Required	Default	Description
attributes	array	optional	[]	Side names or integer boundary attribute IDs

Side names (**top**, **left**, ...) and integer attribute IDs may be mixed. Assigning IDs requires an external mesh tool (CUBIT/GMSH, etc.). For internal meshes, only the domain-boundary sides can be specified using names such as **top**, **bottom**, In SEM, an unspecified boundary becomes a natural boundary condition, so for acoustic wave simulations **dirichlet** is mandatory.

4.8.3 Example settings for the boundary section

```
# Marine simulation: Dirichlet on the sea surface, absorbing on the others
boundary:
  absorbing:
    type: cerjan
    sides: [bottom, left, right]
    thickness: 1200.0
    alpha: 5.0
  dirichlet:
    attributes: [top]
```

4.9 sources section

4.9.1 Source mode and input format

Key	Type	Required	Default	Description
<code>mode</code>	string	required	—	<code>simultaneous</code> or <code>sequential</code>
<code>format</code>	string	optional	<code>yaml</code>	Source information input format: <code>yaml</code> or <code>hdf5</code>
<code>file</code>	string	conditional	—	Path to an external source definition file. Required when <code>format=hdf5</code>
<code>shot_id</code>	int	optional	0	Target shot ID for <code>format=hdf5</code> (≥ 0)
<code>list</code>	array	conditional	—	Inline source definitions (only when <code>format=yaml</code> ; mutually exclusive with <code>format=hdf5</code>)

`simultaneous` fires all sources at once. `sequential` processes the sources one at a time (an in-simulation loop).

format: `yaml` (default). With `file` specified, the `list:` is read from an external YAML file. Without `file`, the inline `sources.list` is used. This is useful when there are many sources or when an FWI workflow generates the source file programmatically.

format: `hdf5`. A path that reads one shot worth of source information from the SEMSWS HDF5 v2.0 schema. The schema, constraints, and usage are gathered in §4.13; see that section.

4.9.2 Common source parameters (list)

Each element of the `sources.list` array takes the following:

Key	Type	Required	Default	Description
<code>id</code>	int	required	—	Source ID
<code>name</code>	string	optional	—	Source name (an inert key in the current version)
<code>type</code>	string	required	—	<code>force</code> , <code>pressure</code> , <code>moment_tensor</code>
<code>location</code>	real array	required	—	Position <code>[x,y]</code> or <code>[x,y,z]</code> (m)

4.9.3 Source type: force

A single force for elastic wave simulations.

Key	Type	Required	Description
direction	real array	required	Direction vector of the force [x,d] or [x,y,z]

```
- id: 1
  type: force
  location: [500.0, 300.0, 0.0]
  direction: [0.0, 0.0, 1.0]      # vertical
  wavelet:
    type: ricker
    frequency: 10.0
    amplitude: 1.0e10
```

4.9.4 Source type: pressure

Pressure point source for acoustic wave simulations. No additional parameters.

4.9.5 Source type: moment_tensor

A moment-tensor source. Specifies an earthquake focal mechanism directly.

In 3D (6 components): Mxx, Myy, Mzz, Mxy, Mxz, Myz.

In 2D (3 components): Mxx, Myy, Mxy.

```
- id: 1
  type: moment_tensor
  location: [5000.0, 5000.0, 3000.0]
  moment_tensor:
    Mxx: 1.0e18
    Myy: 0.0
    Mzz: 0.0
    Mxy: 0.0
    Mxz: 0.0
    Myz: 0.0
  wavelet:
    type: ricker
    frequency: 5.0
    amplitude: 1.0
```

4.9.6 Wavelet (wavelet)

Key	Type	Required	Default	Description
type	string	optional	ricker	ricker, gaussian, external
frequency	real	conditional	—	Dominant frequency (Hz). Required for ricker/-gaussian
amplitude	real	optional	1.0	Amplitude scaling
delay	real	optional	0.0	Delay of the peak time (s)
file	string	conditional	—	Path to an external STF file (required for external)

For the Ricker wavelet, $\text{delay} \geq 1.2/\text{frequency}$ is recommended.

An external STF file is a 2-column ASCII text (time, amplitude). The time column is ignored, so the first row should be aligned with the simulation start time.

```
# time (s)      amplitude
0.0000         0.000000e+00
0.0001         1.234568e-05
...
```

4.9.7 Observed data (observed) — inversion mode

When `simulation.mode` is `inversion` or `misfit_only`, observed data are specified per source via `observed:.`. The observed data reference a single file written in the SEMSWS HDF5 v2.0 schema (`/shots/<NNNN>/receivers/<key>/...`).

Key	Type	Required	Default	Description
<code>observed.file</code>	string	required	—	Path to the observed HDF5 file (.h5)
<code>observed.resample.enabled</code>	bool	optional	false	Whether to resample the observation sampling interval to the simulation's <code>dt</code>
<code>observed.resample.method</code>	string	optional	<code>lanczos</code>	Resampling algorithm: <code>lanczos</code> or <code>linear</code>
<code>observed.resample.lanczos_a</code>	int	optional	4	Lanczos kernel support width (only when <code>method=lanczos</code>)

The legacy SU schema (`observed.format + observed.files[]`) has been removed. To use SU/SEG-Y as observed data, convert them in advance to the SEMSWS HDF5 v2.0 format.

4.9.8 Example settings for the sources section

Acoustic waves (multiple sources, simultaneous firing):

```
sources:
  mode: simultaneous
  list:
    - id: 1
      type: pressure
      location: [300.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
    - id: 2
      type: pressure
      location: [700.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
```

Elastic waves (moment tensor, external STF):

```
sources:
  mode: sequential
```

```
list:
  - id: 1
    type: moment_tensor
    location: [5000.0, 5000.0, 3000.0]
    moment_tensor:
      Mxx: 1.0e18
      Myy: -1.0e18
      Mzz: 0.0
      Mxy: 0.0
      Mxz: 0.0
      Myz: 0.0
    wavelet:
      type: external
      frequency: 5.0
      file: "stf_earthquake.txt"
```

The structure of the external file is identical to the `list` of the `sources` section in `config.yaml`:

```
# config.yaml side
sources:
  mode: simultaneous
  file: "./sources.yaml"

# sources.yaml (external file)
sources:
  - id: 1
    type: pressure
    location: [1800.0, 4990.0]
    wavelet:
      type: ricker
      frequency: 30.0
      amplitude: 1.0e10
      delay: 0.05
  - id: 2
    type: pressure
    location: [2644.0, 4990.0]
    wavelet:
      type: ricker
      frequency: 30.0
      amplitude: 1.0e10
      delay: 0.05
```

4.10 receivers section

4.10.1 Recording type and output settings

Key	Type	Required	Default	Description
<code>type</code>	string/array	required	—	Default recording type (overridable per receiver, see below)
<code>format</code>	string	optional	<code>yaml</code>	Receiver information input format: <code>yaml</code> or <code>hdf5</code>
<code>file</code>	string	conditional	—	<code>format=yaml</code> : external YAML containing <code>list:/line::</code> <code>format=hdf5</code> : HDF5 file (required). See §4.10.4.
<code>shot_id</code>	int	optional	0	Target shot ID for <code>format=hdf5</code> (≥ 0)
<code>output.formats</code>	array	required	—	List of output formats (mapping elements)
<code>output.filename</code>	string	conditional	—	Base file name (required for <code>hdf5/su</code>)

format: hdf5 (receiver locations as input). A path that reads receiver locations and types from the SEMSWS HDF5 v2.0 schema. See §4.13 for the detailed schema and examples. The output side (`output.formats`, `output.filename`) is specified in YAML independently of `format`.

Values allowed in `type`:

Value	Description	Applicable model
DISP	Displacement	elastic
VEL	Velocity	elastic
ACC	Acceleration	elastic
PS	Pressure	acoustic

Output formats. `output.formats` is a list of mapping elements, each of whose `type` key selects one of `ascii` (text, one file per receiver), `hdf5` (single HDF5 file), or `su` (Seismic Unix format). Multiple formats can be requested simultaneously:

```
output:
  formats:
    - type: ascii
    - type: su
  filename: "seismograms"      # required when using hdf5 / su
```

To visualize the resulting SU file with Seismic Unix, a Seismic Unix build with XDR-FLAG=OFF is needed.

Per-receiver type override. Each element of `list` may specify its own `type` key (scalar or array) that overrides the parent `type`. This is especially useful in simulations with `material.type: coupled`: the fluid-side observation points can record only [PS] while the solid-side ones record only [DISP, VEL, ACC]. That avoids the “receiver type not found” error in the corresponding sub-mesh that would otherwise arise from requesting a recording type that does not exist on that side.

Key	Type	Required	Default	Description
<code>name</code>	string	required	—	Receiver name
<code>location</code>	real array	required	—	Position <code>[x,y]</code> or <code>[x,y,z]</code> (m)
<code>type</code>	string/array	optional	Parent <code>receivers.type</code>	Recording type applied to this receiver only
<code>weight</code>	real	optional	1.0	Scalar weight (used when assembling FWI observed data)

4.10.2 Receiver line (line)

Automatically generates a line of receivers placed at equal intervals.

Key	Type	Required	Default	Description
<code>line.start</code>	real array	required	—	Start position (m)
<code>line.end</code>	real array	required	—	End position (m)
<code>line.count</code>	int	required	—	Number of receivers
<code>line.prefix</code>	string	required	—	Name prefix

The name is formed by appending a 3-digit number to `prefix` (e.g., `prefix: "REC"` → `REC001`, `REC002`, ...).

4.10.3 Example settings for the receivers section

List form (individual specification):

```
receivers:
  type: [DISP, VEL]
  output:
    formats:
      - type: ascii
      - type: su
    filename: "seismograms"
  list:
    - name: "R01"
      location: [1000.0, 2500.0, 0.0]
    - name: "R02"
      location: [2000.0, 2500.0, 0.0]
    - name: "R03"
      location: [3000.0, 2500.0, 0.0]
```

Line form (automatic equal-interval generation):

```
receivers:
  type: [PS]
  output:
    formats:
      - type: ascii
  line:
    start: [100.0, 300.0]
    end: [900.0, 300.0]
    count: 17
    prefix: "REC"
```

Per-receiver type override example (a typical fluid–solid coupling layout where R001 is on the fluid side and R002/R003 are on the solid side, each recording the physically appropriate type):

```
receivers:
  output:
    formats:
      - type: ascii
  type: [PS, DISP, VEL, ACC]  # the union of all types requested below
  list:
    - name: "R001"
      location: [800.0, 200.0]
      type: [PS]
    - name: "R002"
      location: [800.0, 440.0]
      type: [DISP, VEL, ACC]
    - name: "R003"
      location: [800.0, 680.0]
      type: [DISP, VEL, ACC]
```

4.10.4 External receiver file (receivers.file)

When there are many receivers, they can be split out from `config.yaml` into an external YAML file referenced via the `receivers.file` key. The external file has `list:` / `line:` at the top level, with the same schema as the inline specification. `type`, output formats, and the file name remain in the main configuration.

```
# config.yaml
receivers:
  type: [PS]
  output:
    formats:
      - type: ascii
  file: "./receivers.yaml"  # overrides any list/line written below
```

```
# receivers.yaml - path is resolved from the current directory
list:
  - name: "REC001"
    location: [100.0, 300.0]
  - name: "REC002"
    location: [200.0, 300.0]
  - name: "REC003"
    location: [300.0, 300.0]

# line: may also be combined with list:
# line:
#   start: [0.0, 300.0]
#   end:   [1000.0, 300.0]
#   count: 21
#   prefix: "LINE"
```

When `receivers.file` is specified, any `list:` / `line:` written inline in the main configuration is ignored; the external file takes precedence. If the external file is missing or cannot be parsed (YAML syntax error, etc.), execution is aborted with a message describing the cause.

4.11 device section

Key	Type	Required	Default	Description
<code>type</code>	string	optional	<code>cpu</code>	<code>cpu</code> , <code>cuda</code> , <code>hip</code>
<code>seismo_buffer_steps</code>	int	optional	0	Receiver waveform buffer size (GPU only)

`type` must match the build configuration. Specifying `cpu` on a GPU build runs but is very slow, so a CPU build should be used for CPU execution.

`seismo_buffer_steps` specifies the number of steps for the receiver waveform buffer on the GPU. When 0, all steps are buffered in GPU memory (fastest, but memory-heavy); when $N > 0$, the buffer is transferred to the host every N steps.

4.11.1 Example settings for the device section

```
device:
  type: cuda
  seismo_buffer_steps: 1000
```

4.12 inversion section

An optional top-level section, valid when `simulation.mode` is `inversion` or `misfit_only`. It switches the sensitivity-kernel computation path.

Key	Type	Required	Default	Description
<code>inversion.sensitivity.backend</code>	string	optional	<code>hand</code>	Sensitivity-kernel backend: <code>hand</code> (the hand-written adjoint path used in production) or <code>ad</code> (a forward-mode automatic-differentiation path for verification).

`ad` is an alternate path used to verify numerical agreement with the hand-written implementation (`hand`), and is computationally expensive. Normally the default `hand` should be left as is.

```
inversion:
  sensitivity:
    backend: hand          # or "ad" for AD verification
```

4.13 HDF5 I/O Schema and Usage

In addition to YAML, SEMSWs supports an HDF5 v2.0 schema path for inputting sources, receivers, and observed data, and for outputting receiver waveforms. For multi-shot FWI workflows (where `semsws-driver` loops on the outside) and for the inversion mode that reads observed data, HDF5 is effectively the required input format.

The schema itself is shared between input and output, but the **input path** and the **output path** differ in what SEMSWS reads from / writes to the file. This section describes the two clearly separately.

4.13.1 Overview of the HDF5 v2.0 schema (common)

Every input/output file has the root attribute `format_version="2.0"` and collects, under `/shots/<NNNN>/` (where `<NNNN>` is a 4-digit zero-padded shot ID), the sources and receivers for one shot.

<code>/</code>	attrs: <code>format_version="2.0",</code> <code>dt, t0, n_samples, space_dim,</code> <code>coord_system="cartesian", units="</code> <code>SI"</code>
<code>/shots/<NNNN>/</code>	attrs: <code>shot_id (i32)</code>
<code> /sources/<key>/</code>	attrs: <code>id (i32), type (str),</code> <code> position (f32[space_dim]),</code> <code> direction (f32[space_dim]; for</code> <code> force),</code> <code> moment_tensor (f32[6 or 3])</code> <code>(n_samples,) f32 # STF</code>
<code> /stf</code>	
<code> time series</code>	
<code> /receivers/<key>/</code>	attrs: <code>position (f32[space_dim])</code> <code> types (var-length str array;</code> <code> optional)</code> <code> label (str; optional)</code>
<code> /pressure</code>	<code>(nt,) f32 #</code>
<code> waveform for type=PS</code>	
<code> /velocity</code>	<code>(space_dim, nt) f32 #</code>
<code> waveform for type=VEL</code>	
<code> /displacement</code>	<code>(space_dim, nt) f32 #</code>
<code> waveform for type=DISP</code>	
<code> /acceleration</code>	<code>(space_dim, nt) f32 #</code>
<code> waveform for type=ACC</code>	
<code> /weight_<ch></code>	<code>same shape as <ch> f32 #</code>
<code> optional, observed-side weights</code>	

Conventions for group names `<key>`.

- **Source:** `S<NNNN>` (4-digit zero-padded). Must match the `@id` attribute (e.g., `@id=42` requires `S0042`). The reader strictly validates this.
- **Receiver:** any string. No naming convention. If a `@label` attribute is present, it is used as the user-visible display name; otherwise, the group name itself is the display name.

The canonical headers on the C++ side reside in `include/srcrecv/HDF5IOSchema.hpp`, and on the Python driver side in `driver/src/semsws_driver/io/hdf5_schema.py`; the attribute and dataset name constants are centralized in both and kept in sync.

4.13.2 Input path: HDF5 -> SEMSWS

Passing sources via HDF5 (`sources.format: hdf5`)

```
sources:
  mode: simultaneous           # see \S\ref{subsubsec:hdf5-input-shot-loop}
  format: hdf5
  file: inputs/shots.h5
```

```
shot_id: 0                                # reads /shots/0000/sources/
```

`sources.shot_id` specifies the shot number to read (mapped to a 4-digit zero-padded group name). The source type / position / direction / moment_tensor and STF time series are all taken from HDF5, so the YAML `sources.list` must not be written (specifying it triggers a mutually-exclusive validator error).

Important: in the `format: hdf5` path, the STF **must be stored as time series** in the `/shots/<NNNN>/sources/<key>/stf` dataset. Analytic-wavelet specifications such as `wavelet.type: ricker` via YAML cannot be combined with `format: hdf5` (a YAML wavelet only works on the YAML `list: input` path). To use a Ricker etc. wavelet, precompute the corresponding discrete time series and write it into the HDF5 file in advance (this can be generated with the utilities bundled with `semsws-driver`).

Passing receiver locations via HDF5 (`receivers.format: hdf5`)

```
receivers:
  format: hdf5
  file: inputs/shots.h5
  shot_id: 0                                # reads /shots/0000/receivers/
  type: [PS]                                # <- default type for all receivers (
    overridable per-receiver in HDF5)
  output:
    formats: [{type: ascii}]
    filename: "seis"
```

When `format=hdf5` is specified, `list:/line:` are ignored (the receiver set in the HDF5 file is adopted as is).

Rules for deciding each receiver’s recording type (important). The recording type of each receiver is determined in this priority order:

1. If the **receiver group in HDF5** has a `@types` attribute, it is used (a var-length string array, e.g., `["DISP", "VEL"]`).
2. Receivers without a `@types` attribute fall back to the **YAML `receivers.type`** as the default.
3. If the **YAML `receivers.type`** is also absent, the validator rejects the input.

Example: three receivers in HDF5, with R001 having `@types=["PS"]`, R002 having `@types=["DISP", "VEL"]` and R003 having no `@types`, and **YAML `receivers.type: [PS, DISP, ACC]`**:

- R001 → [PS]
- R002 → [DISP, VEL]
- R003 → [PS, DISP, ACC] (default from YAML)

This allows the “fluid-side receivers record PS and solid-side ones record DISP/VEL/ACC” separation in a fluid–solid coupling simulation to be encoded directly in the HDF5 file. Keeping the **YAML `receivers.type`** as the union (`[PS, DISP, VEL, ACC]`) ensures that only the physically meaningful types are applied to each receiver.

Observed data (inversion only)

For `simulation.mode: inversion` or `misfit_only`, the observed waveform for each source is specified as:

```
sources:
  list:
    - id: 1
      type: pressure
      location: [...]
      wavelet: { type: ricker, frequency: 5.0, amplitude: 1.0, delay: 0.4 }
      observed:
        file: obs/shot_0000.h5          # required
        resample:
          enabled: true
          method: lanczos                # or "linear"
          lanczos_a: 4
```

The observed file references datasets under `/shots/<NNNN>/receivers/<key>/` (pressure, velocity, displacement, acceleration) per the schema in §4.13.1. If the receiver `@position` attribute does not match the receiver location on the simulation side, the misfit computation will be off.

The legacy SU schema (`observed.format + observed.files[]`) has been removed. To use SU/SEG-Y as observed data, convert them in advance to the SEMSWHS HDF5 v2.0 format.

Inter-shot loop and the responsibility of mode

With HDF5 input, each SEMSWHS launch processes **only a single shot** (selected by `shot_id`).

- **Inter-shot loop** (process `/shots/0000`, `/shots/0001`, ...in order): the responsibility of `semsws-driver`. It separates `output.directory` for each shot and can submit them as independent parallel jobs.
- **Intra-shot loop** (multiple sources `S0001`, `S0042`, ... inside the same `shot_id`): controlled by `sources.mode`.
 - `mode: simultaneous` (**recommended**): all sources within the shot are fired at once, in a single simulation. Matches the typical “1 shot = 1 physical event” design of FWI.
 - `mode: sequential`: runs the simulation M times for M sources within the shot, saving each contribution to a separate file. Reserved for research purposes (per-source sensitivity-kernel comparisons, debugging, etc.).

Examples of input HDF5 structure

Three typical input HDF5 layouts are illustrated below by use case. In all of them, the root attributes (`format_version=2.0`, `dt`, `t0`, `n_samples`, `space_dim`, ...) are omitted for brevity.

Example 1: single source, single shot (typical ocean-bottom seismometer simulation).

```
inputs/shot.h5
+- /shots/0000/                                attrs: shot_id=0
| - /sources/
|   +- /S0001/                                attrs: id=1, type="pressure",
|       |                                     position=[5000, 4000]
|       +- /stf                               (n_samples,) <- precomputed time
|           series
+- /receivers/
```

```

|- /R001/  attrs: position=[1000, 0]
|- /R002/  attrs: position=[2000, 0]
+- /R003/  attrs: position=[3000, 0]

```

Corresponding YAML:

```

sources:
  mode: simultaneous
  format: hdf5
  file: inputs/shot.h5
  shot_id: 0
receivers:
  format: hdf5
  file: inputs/shot.h5
  shot_id: 0
  type: [PS]                                     # PS applied to all receivers

```

Example 2: multiple shots with multiple sources per shot (multi-source FWI).

```

inputs/multishot.h5
|- /shots/0000/                                attrs: shot_id=0
| |- /sources/
| | |- /S0001/  attrs: id=1, type="force", direction=[1,0,0]
| | | +- /stf
| | +- /S0002/  attrs: id=2, type="moment_tensor",
| | |           moment_tensor=[Mxx,Myy,Mzz,Mxy,Mxz,Myz]
| | | +- /stf
| +- /receivers/
|   |- /STN_001/  attrs: position=[...], types=["DISP","VEL"]
|   +- /STN_002/  attrs: position=[...], types=["VEL"]
|- /shots/0001/                                attrs: shot_id=1
| |- /sources/
| | +- /S0010/  attrs: id=10, type="pressure"
| | | +- /stf
| +- /receivers/
|   |- /STN_001/  attrs: position=[...], types=["DISP","VEL"]
|   +- /STN_002/  attrs: position=[...], types=["VEL"]
+- /shots/0002/                                attrs: shot_id=2
...

```

The driver invokes SEMSWs with `shot_id=0,1,2,...` in turn. Within each shot, multiple sources are fired simultaneously in `mode: simultaneous`. The per-receiver `@types` attribute lets each station record a different set of channels.

Example 3: fluid-solid coupling (receiver types split by physics).

```

inputs/fluid_solid.h5
+- /shots/0000/                                attrs: shot_id=0
| |- /sources/
| | +- /S0001/  attrs: id=1, type="pressure", position=[3000, 200]
| | | +- /stf                                     (an acoustic source placed in the
| | | fluid)
| +- /receivers/
|   |- /OBS_FLUID_01/                            <- fluid-side receiver
|   |   attrs: position=[1000, 100], types=["PS"]
|   |- /OBS_FLUID_02/
|   |   attrs: position=[2000, 100], types=["PS"]
|   |- /OBS_SOLID_01/                            <- solid-side receiver
|   |   attrs: position=[1500, 800], types=["DISP", "VEL"]
|   +- /OBS_SOLID_02/

```

```
attrs: position=[2500, 800], types=["DISP", "VEL"]
```

Corresponding YAML (here, `receivers.type` is set to the union and the physical consistency is enforced by per-receiver `@types` overrides on the HDF5 side):

```
receivers:
  format: hdf5
  file: inputs/fluid_solid.h5
  shot_id: 0
  type: [PS, DISP, VEL]      # union; overridden by each receiver's @types
```

4.13.3 Output path: SEMSWS -> HDF5

HDF5 output of receiver waveforms (`output.formats: hdf5`)

Adding `type: hdf5` to `receivers.output.formats` saves the simulation's receiver waveforms in HDF5 under the schema of §4.13.1. `output.filename` (the `.h5` extension may be omitted) becomes the base name, and a source ID suffix is appended as described below.

```
receivers:
  type: [PS]
  output:
    formats: [{type: hdf5}]
    filename: "seis"          # -> seis<NNNN>.h5
  list:
    - { name: "R001", location: [1000, 500] }
    - { name: "R002", location: [2000, 500] }
```

(ascii, su, and other formats can be combined; their naming and structure are described in §4.10.1. The following discussion focuses on HDF5.)

For the receiver group names (`<key>`) in the output file, **the YAML `list[].name` (or `line.prefix + 3-digit number`) is used directly as the group name** (unlike on the reader side, the writer does not convert receiver names). In the example above, the output is, e.g., `seis<NNNN>.h5:/shots/0000/receivers/R001/`.

The source-side group is named automatically following the `S<NNNN>` convention.

Correspondence between output file name and internal shot ID

The naming of the receiver output files produced by a single SEMSWS invocation and the internal `/shots/<NNNN>/` key depend on mode.

mode: simultaneous (recommended). Regardless of the number of sources inside the shot, **exactly one set per output format** is produced (the combined waveform of all sources is saved in a single file). Both the file-name suffix (4-digit zero-padded) and the internal `/shots/<NNNN>/` key adopt the **input sources.shot_id**.

Example: input `sources.shot_id=42`, with sources `S0001`, `S0042`, `S0100` inside shot 42 fired simultaneously:

```
out/seis0042.h5          # exactly one file, suffix = input shot_id = 42
```

Contents of `seis0042.h5` (self-roundtrip spec, see §4.13.3):

```
seis0042.h5
|- root attrs: format_version=2.0, dt, t0, n_samples, ...
+- /shots/0042/          attrs: shot_id=42 <- matches input
  |- /sources/           <- metadata for all active sources
    | |- /S0001/ (id=1, type, position, /stf, ...)
```



```

|   |- /S0042/   (id=42, ...)
|   +- /S0100/   (id=100, ...)
+- /receivers/                                     <- combined waveforms for all
   receivers
+- /R001/   (position, /pressure, /velocity, ...)

```

Because the input `shot_id` is reflected in both the file name and the internal key, the file alone makes it obvious “which input shot this is the result of.”

mode: sequential. For M sources within the shot, M **simulations run, and M files per format** are produced. Each file is the output when only one source (the active source for that iteration) was fired. The file-name suffix is the **source.@id** of each iteration (needed to distinguish the M files). The internal `/shots/<NNNN>/` key is the input `shot_id` (shared across all iterations).

Example: input `shot_id=42`, with sources S0001, S0005, S0009 in the shot:

```
out/seis0001.h5, out/seis0005.h5, out/seis0009.h5    # 3 HDF5 files
```

Each HDF5 file contains **only the result of a single source firing** (not a sum):

```

seis0001.h5                                     <- waveforms with only S0001 firing
|- root attrs: ...
+- /shots/0042/                                attrs: shot_id=42   <- matches input shot_id
   |- /sources/
   |   +- /S0001/                               <- only one active source
+- /receivers/
   +- /R001/   (...)

seis0005.h5                                     <- waveforms with only S0005 firing
+- /shots/0042/                                attrs: shot_id=42   <- another iteration of
   the same shot
   |- /sources/
   |   +- /S0005/                               <- only one active source
+- /receivers/
   +- /R001/   (...)

seis0009.h5                                     <- similarly
+- /shots/0042/
   |- /sources/
   |   +- /S0009/
+- /receivers/
   +- /R001/   (...)

```

All three HDF5 files have internal `/shots/0042/` (matching the input `shot_id`). The distinction between files is made by the **file-name suffix** (source.id: 0001, 0005, 0009).

Self-roundtrip spec and driver merge

Self-roundtrip. Each output file `seis<NNNN>.h5` contains both `/sources/` and `/receivers/`, so it can be reused directly as an input to SEMSW. For example, forward results can be passed as observed data to the next inversion, or used as the reference for a forward reproducibility test, and so on.

How the correspondence to the input shot is preserved. The internal `/shots/<NNNN>/` key and the `@shot_id` attribute are written to match the input `sources.shot_id`. Additionally, in the simultaneous mode, the file-name suffix also matches `shot_id`, so the **output file alone makes it obvious “which input shot this is the result of.”**

When `semsws-driver` is used, the driver constructs the per-shot directory layout automatically:

```
output_root/
|- shot_0000/                                <- result of input shot 0
|  |- config.yaml                            (YAML rewritten for shot 0, shot_id=0)
|  |- seismograms0000.h5                    (internal /shots/0000/, suffix also
      0000)
|  +- stdout.log
|- shot_0001/                                <- input shot 1
|  |- config.yaml                            (shot_id=1)
|  |- seismograms0001.h5                    (internal /shots/0001/, suffix 0001)
|  +- stdout.log
+- shot_0002/
   |- config.yaml                            (shot_id=2)
   |- seismograms0002.h5                    (internal /shots/0002/, suffix 0002)
   +- stdout.log
```

`shot_dir(shot_id)` in `driver/src/semsws_driver/core/layout.py` names the directory, and it is also recorded in the driver's manifest. When generating the per-shot YAML, the driver sets `sources.shot_id` and `receivers.shot_id` to the input shot and sets `receivers.output.filename` to "seismograms" (fixed) (`driver/src/semsws_driver/core/template.py`). As a result, the shot identity is expressed three times—through the directory name `shot_<NNNN>`, the file-name suffix `<NNNN>`, and the internal `/shots/<NNNN>/` key.

Even without the driver, when invoking SEMSWs manually for each shot, simply changing `sources.shot_id` per invocation automatically separates the file name and the internal key, so there are no collisions:

```
for SHOT in 0 1 2; do
  yq -i ".sources.shot_id = $SHOT |
        .simulation.output.directory = \"./out/shot_$(printf %04d $SHOT)\"
        \" \
        config.yaml
  mpirun -np 8 semsws --config config.yaml
done
```

Each shot produces `seis0000.h5`, `seis0001.h5`, `seis0002.h5` in separate directories. Even without separating `output.directory`, the file-name suffix differentiates them, but logs and configs would mix between shots, so separating the directory is recommended.

driver's merge tool. When `semsws-driver` processes multiple shots in turn, a post-processing script is provided that merges the per-shot files into a single HDF5 (equivalent of `driver/tools/merge_shot_out`). Example output:

```
shots_merged.h5
+- /shots/
   |- /0000/ (sources + receivers for shot 0)
   |- /0001/ (... for shot 1)
   +- /0002/ (...)
```

Because each per-shot file already has `/shots/<NNNN>/` at the correct index, the merge tool does not need to renumber `shot_id`—it simply concatenates them.

Examples of output configurations (YAML config -> resulting HDF5)

Example 1: single source, simultaneous (typical forward simulation). YAML:

```
sources:
  mode: simultaneous
  shot_id: 0          # defaults to 0 if omitted
  list:
    - id: 1
      type: pressure
      location: [5000, 4000]
      wavelet: { type: ricker, frequency: 5.0, amplitude: 1.0, delay: 0.4 }
receivers:
  type: [PS]
  output:
    formats: [{type: hdf5}]
    filename: "seis"
  list:
    - { name: "R001", location: [1000, 0] }
    - { name: "R002", location: [2000, 0] }
```

Generated file (one):

```
out/seis0000.h5          <- suffix = input shot_id = 0
+- /shots/0000/          attrs: shot_id=0
  |- /sources/
  | +- /S0001/  attrs: id=1, type="pressure", position=[5000,4000]
  |   +- /stf              (n_samples,)
+- /receivers/
  |- /R001/  attrs: position=[1000,0]
  |   +- /pressure          (nt,)
  +- /R002/  attrs: position=[2000,0]
    +- /pressure          (nt,)
```

Example 2: multiple sources, simultaneous (simultaneous firing, combined waveforms in one file). YAML:

```
sources:
  mode: simultaneous      # <- fire all sources at once
  shot_id: 7              # <- assume input shot_id=7
  list:
    - id: 1; type: force; location: [...]; direction: [1,0]; wavelet: {...}
    - id: 2; type: force; location: [...]; direction: [0,1]; wavelet: {...}
    - id: 3; type: pressure; location: [...]; wavelet: {...}
receivers:
  type: [DISP, VEL]
  output:
    formats: [{type: hdf5}]
    filename: "seis"
  list:
    - { name: "STN_001", location: [...] }
```

Generated file (one HDF5 containing the combination of all 3 sources):

```
out/seis0007.h5          <- suffix = input shot_id = 7
+- /shots/0007/          attrs: shot_id=7
  |- /sources/            <- metadata for all active
    sources
  |   |- /S0001/  (force, /stf)
  |   |- /S0002/  (force, /stf)
  |   +- /S0003/  (pressure, /stf)
+- /receivers/
```

```

+- /STN_001/                                <- combined waveform from 3
  sources
  |- /displacement                          (space_dim, nt)
  +- /velocity                             (space_dim, nt)

```

Important: even with 3 sources, only 1 HDF5 file is produced. Each receiver's waveform dataset contains the combined contribution from all 3 sources. The file-name suffix and the internal `/shots/<NNNN>/` both reflect the input `shot_id`.

Example 3: multiple sources, sequential (per-source separate outputs). YAML:

```

sources:
  mode: sequential                          # <- M iterations, fire each source
      individually
  shot_id: 42                              # <- assume input shot_id=42
  list:
    - id: 1; type: pressure; location: [...]; wavelet: {...}
    - id: 5; type: pressure; location: [...]; wavelet: {...}
    - id: 9; type: pressure; location: [...]; wavelet: {...}
receivers:
  type: [PS]
  output:
    formats: [{type: hdf5}]
    filename: "seis"
  list:
    - { name: "R001", location: [...] }

```

Generated files (3 HDF5 files, each from a single source firing. **The suffix in sequential mode is the source.id of each iteration**):

```

out/seis0001.h5                                <- S0001 firing alone, suffix=
  source.id=1
+- /shots/0042/                                attrs: shot_id=42 (matches input)
  |- /sources/
  |   +- /S0001/ (id=1, /stf)                  <- only one active source
+- /receivers/
  +- /R001/
      +- /pressure                             (nt,) <- contribution from S0001
      only

out/seis0005.h5                                <- S0005 firing alone
+- /shots/0042/                                attrs: shot_id=42 (same shot)
  |- /sources/
  |   +- /S0005/ (id=5, /stf)
+- /receivers/
  +- /R001/
      +- /pressure                             <- contribution from S0005 only

out/seis0009.h5                                <- S0009 firing alone
+- /shots/0042/                                attrs: shot_id=42
  |- /sources/
  |   +- /S0009/ (id=9, /stf)
+- /receivers/
  +- /R001/
      +- /pressure                             <- contribution from S0009 only

```

All 3 HDF5 files share the internal `/shots/0042/` (matching the input `shot_id`). The files are distinguished by **the source.id suffix in the file name**. Each file is self-roundtrip-capable (it can be reused as the input to a subsequent inversion, etc.).

Chapter 5

Tools and Scripts

In addition to the main simulation binary (**semsws**), SEMSWS comes with preprocessing tools for mesh partitioning, refinement, and quality evaluation, along with Python scripts for visualizing the results. This chapter describes how to use them.

5.1 `partition_mesh` — Pre-partition a mesh

`partition_mesh` pre-partitions a mesh file for MPI-parallel execution (serial run only). Using a pre-partitioned mesh skips the mesh load + partition overhead at the start of each simulation. It is useful when running many simulations with the same mesh and partition count.

5.1.1 Usage

```
# Partition an external mesh into 64 parts with METIS
./build/src/partition_mesh -n 64 domain.exo partitions/

# Cartesian partitioning (4*4*4 = 64)
./build/src/partition_mesh -n 64 -p cartesian -g 4,4,4 domain.exo
    partitions/

# Generate an internal mesh and partition it
./build/src/partition_mesh -n 64 --internal \
    --size 10000,10000,5000 --elements 100,100,50 partitions/
```

5.1.2 Options

Option	Description
<code>-n <nparts></code>	Number of partitions (required)
<code>-p <method></code>	Partitioning method: metis (default) or cartesian
<code>-g <nx,ny,nz></code>	Grid count for Cartesian partitioning
<code>--internal</code>	Internal mesh generation mode
<code>--dim <2 3></code>	Dimension (for internal mesh; default 3)
<code>--origin <x,y,z></code>	Origin (for internal mesh)
<code>--size <lx,ly,lz></code>	Domain size (for internal mesh; required)
<code>--elements <nx,ny,nz></code>	Number of elements (for internal mesh; required)

The output files are produced in the specified directory as `mesh.mesh.000000`, `mesh.mesh.000001`, ...

5.1.3 Use in config.yaml

To use the pre-partitioned mesh in the simulation, configure the `mesh` section of `config.yaml` like this:

```
mesh:
  type: partitioned
  partitioned:
    directory: "partitions/"
    nparts: 64
```

`nparts` must match the MPI process count at runtime.

5.2 combine_mesh — Combine partitioned mesh

`combine_mesh` merges a mesh partitioned by `partition_mesh` into a single file. It is used for visualization with GLVis or to inspect the partitioned mesh.

```
# Combine a 64-way partitioned mesh (run with np equal to the partition
  count)
mpirun -np 64 ./build/src/combine_mesh partitions/ combined.mesh
```

The MPI process count at runtime must match the number of partitions.
The combined mesh can be checked with GLVis:

```
glvis -m combined.mesh
```

5.3 evaluate_mesh — Mesh quality evaluation

`evaluate_mesh` evaluates the quality of a mesh and verifies consistency with the simulation parameters. It computes statistics on element size, the CFL condition, PPW (Points Per Wavelength), aspect ratio, and Jacobian determinant.

5.3.1 Usage

```
# Basic evaluation (console output only)
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml

# Also output histogram CSV files
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist --nbins 50
```

5.3.2 Options

Option	Description
<code>--config <file></code>	Configuration file (required)
<code>--cfl <factor></code>	Override the CFL factor
<code>--histogram <prefix></code>	Output prefix for the histogram CSV files
<code>--nbins <N></code>	Number of histogram bins (default: 50)

5.3.3 Histograms produced

Specifying `--histogram` produces the following CSV files:

File	Contents
<code><prefix>_element_size.csv</code>	Element size distribution
<code><prefix>_dt_cfl.csv</code>	Distribution of $\Delta t_{\max}$ from the CFL condition
<code><prefix>_ppw.csv</code>	PPW distribution
<code><prefix>_aspect_ratio.csv</code>	Aspect ratio distribution
<code><prefix>_jacobian_det.csv</code>	Jacobian determinant distribution

5.4 `refine_mesh` — Uniform mesh refinement and partitioning

`refine_mesh` uniformly refines a mesh based on the material properties. It automatically subdivides elements in low-velocity regions (where the wavelength is short) to produce a mesh that satisfies the PPW criterion.

The refinement criterion is determined as follows. First, the required element size h_{required} is computed:

$$h_{\text{required}} = \frac{v_{\min} \times N_{\text{GLL}}}{f_{\max} \times \text{PPW}}$$

where $v_{\min}$ is the minimum wave velocity across all elements, $N_{\text{GLL}} = \text{order} + 1$, and $f_{\max}$ and PPW are specified in the configuration as `mesh.max_freq` and `mesh.ppw`.

Next, the number of refinement levels n is determined from the current element size h_{current} :

$$n = \left\lceil \log_2 \frac{h_{\text{current}}}{h_{\text{required}}} \right\rceil$$

If $h_{\text{current}} \leq h_{\text{required}}$, no refinement is needed ($n = 0$). The maximum n over all elements becomes the refinement level of the entire mesh.

Note that the refinement is applied uniformly to the entire mesh. Each single refinement step bisects each element, so the number of elements grows by 2^{2n} in 2D and 2^{3n} in 3D, which means some compromise on metrics such as PPW is sometimes needed.

There are two main use cases. The first is to produce meshes that satisfy PPW. The second concerns large-scale HPC computations (large meshes). **In SEMSWS, even in a parallel run (except when `mesh.type.partitioned` is set in `config.yaml`), the full mesh is loaded on rank 1, partitioned afterwards, and distributed via `MPI_SEND`. Consequently, for very large meshes, memory exhaustion can occur. Meshes pre-partitioned by tools such as CUBIT cannot be read either. The `partition_mesh` pre-partitioning also begins by loading the entire mesh on one rank.** In such cases, one approach is to create a relatively coarse mesh that captures the geometry sufficiently (topography or bathymetry) in CUBIT or GMSH, and then refine it with `refine_mesh`. As described below, the `refine_mesh` pipeline is: load the input mesh on rank 1 → partition the mesh → refine on each rank → output a partitioned mesh. The produced mesh can then be referenced as `mesh.type.partitioned` in the configuration so that each rank reads only its own partition.

5.4.1 Behavior of mesh loading and output

`refine_mesh` loads the mesh internally via `LoadParMesh()`. The loading path depends on `mesh.type` in the configuration:

Unpartitioned mesh (type: `internal` / `external`) Only rank 0 loads (or, for `internal`, generates) the serial mesh, `MeshPartitioner` splits it into N parts, and then distributes them to each rank with `MPI_Send`. N is determined by the process count of `mpirun -np N`.

Partitioned mesh (type: partitioned) Each rank independently loads its own partition file (`mesh.mesh.NNNNNN`). The MPI process count at runtime must match `nparts`.

Refinement and output In both cases the refinement itself runs in parallel via `ParMesh::UniformRefinement` and the behavior is identical. Only the loading path differs. Two output forms are supported after refinement:

- `--output-dir`: each rank writes its own partition file via `ParPrint()` (N files in total). Used in the simulation with `type: partitioned`.
- `--save-as-one`: rank 0 merges everything into a single file via `SaveAsOne()`. Used for visualization with GLVis or in the simulation with `type: external`.

5.4.2 Usage

```
# Run with 8 processes (MPI required)
mpirun -np 8 ./build/src/refine_mesh --config config.yaml --output-dir MESH
/

# Also save a combined mesh for visualization
mpirun -np 8 ./build/src/refine_mesh --config config.yaml \
  --output-dir MESH/ --save-as-one refined.mesh
```

5.4.3 Options

Option	Description
<code>--config <file></code>	Configuration file (required)
<code>--output-dir <dir></code>	Destination of the partitioned mesh (required)
<code>-n <max_ref></code>	Override the maximum refinement level
<code>--save-as-one <file></code>	Also save a combined mesh (for GLVis visualization)

The output has the same format as `partition_mesh` (`mesh.mesh.NNNNNN`) and can be used directly as `type: partitioned` in `config.yaml`.

5.5 Workflow example: Mesh evaluation and refinement

The following walks through the workflow—from mesh evaluation through refinement to re-evaluation—using the two-layer viscoelastic model in `examples/visco_elastic_3d_layered/` as an example.

5.5.1 Problem setup

A 3D domain of $10\text{ km} \times 10\text{ km} \times 10\text{ km}$ is set up with a two-layer viscoelastic medium. The upper layer ($z > -4000\text{ m}$) is low velocity ($V_p = 3000\text{ m/s}$, $Q_\kappa = Q_\mu = 50$), and the lower layer ($z \leq -4000\text{ m}$) is high velocity ($V_p = 6000\text{ m/s}$, $Q_\kappa = Q_\mu = 100$).

The mesh is generated with CUBIT (the free Coreform version is acceptable). Running the bundled journal file `mesh_layered_3d.jou` produces a mesh in the ExodusII format:

```
# In the CUBIT GUI CONSOLE, run:
playback 'mesh_layered_3d.jou'
```

Contents of the material file `materials_visco.txt`:

#	attribute	<i>vp</i>	<i>vs</i>	<i>rho</i>	<i>qkappa</i>	<i>qmu</i>
1		6000.0	3500.0	2700.0	100.0	100.0
2		3000.0	2000.0	2200.0	50.0	50.0

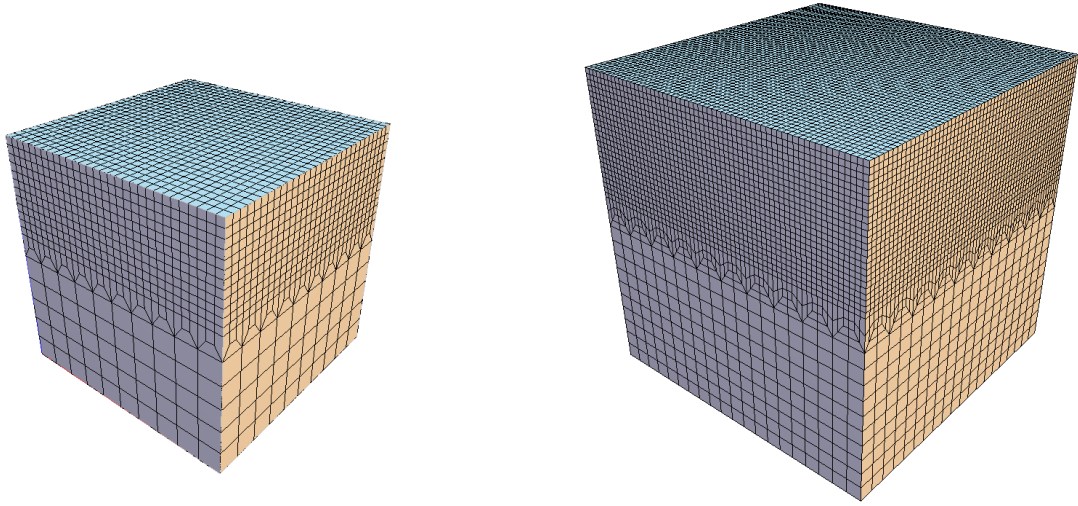


Figure 5.1: Mesh before (left) and after (right) refinement. Left: the initial mesh generated by CUBIT (12,600 elements). The upper layer has 500 m elements and the lower layer has 1,000 m elements. Right: after one uniform refinement with `refine_mesh` (100,800 elements). Every element has been subdivided into $2^3 = 8$ pieces.

5.5.2 Step 1: Evaluating the initial mesh

First evaluate the quality of the initial mesh:

```
./build/src/evaluate_mesh --config config_visco.yaml --histogram mesh
```

Example output:

```
Mesh Evaluation Report
=====
Mesh: 12600 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:
      min      max      mean      std
h [m]      3.333e+02  1.000e+03  3.858e+02  1.561e+02
dt_CFL [s]  2.141e-03  8.562e-03  5.394e-03  1.227e-03
PPW         3.163e+00  9.490e+00  5.436e+00  7.247e-01
Aspect ratio 1.000e+00  8.492e+00  1.331e+00  1.191e+00
det(J)      2.778e+07  1.000e+09  7.540e+07  1.883e+08

CFL summary:
  dt_max (global): 2.1406e-03 s

PPW summary:
  Worst PPW: 3.16 at centroid (9500.00, 4500.00, -6500.00)
  PPW < 5.0: 1300 / 12600 (10.32%)

Per-attribute breakdown:
  Attr 1: 1800 elems, Vp=6050, Vs=3543, PPW_min=3.2, dt_min=2.14e-03
  Attr 2: 10800 elems, Vp=3066, Vs=2049, PPW_min=5.5, dt_min=5.63e-03
```

The minimum PPW is 3.16, below the target of 5, and 1300 elements (10.3%) of Attr 1 (the lower layer, high-velocity region) are PPW-deficient.

5.5.3 Step 2: Uniform refinement

Use `refine_mesh` to refine the mesh. The example below uses 4 processes; the bundled `run_visco.sh` uses 64 processes:

```
mpirun -np 4 ./build/src/refine_mesh \
  --config config_visco.yaml \
  --output-dir mesh/
```

Example output:

```
Loading mesh...
  Elements (global): 12600

Loading material...
  V_min (global): 2049.22
  V_max (global): 6050.01

Refinement levels needed: 1
  Refinement 1: 100800 elements (global)

Refined mesh:
  Elements (global): 100800
  h range:           [84.12, 500.03]
  PPW (worst):       6.33 (target: 5)
  CFL dt_max:        0.00107 s (CFL=0.3)

Saving partitioned mesh to: mesh/
  4 partition files (mesh.mesh.NNNNNN)
```

One refinement step increases the element count from 12,600 to 100,800 (an $8\times$ increase), and the minimum PPW improves from 3.16 to 6.33 (Figure 5.1).

5.5.4 Step 3: Re-evaluating after refinement

Re-evaluate the refined mesh and confirm that the PPW criterion is satisfied. `config_visco_sim.yaml` references the refined mesh via `mesh.type: partitioned`:

```
mpirun -np 4 ./build/src/evaluate_mesh --config config_visco_sim.yaml
```

Example output:

```
Mesh Evaluation Report
=====
Mesh: 100800 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:
```

	min	max	mean	std
h [m]	1.667e+02	5.000e+02	1.938e+02	8.145e+01
dt_CFL [s]	1.070e-03	4.281e-03	2.718e-03	5.622e-04
PPW	6.326e+00	1.898e+01	1.087e+01	1.563e+00
Aspect ratio	1.000e+00	8.495e+00	1.230e+00	8.541e-01

```

PPW summary:
  Worst PPW: 6.33 at centroid (250.00, 9250.00, -9250.00)
  PPW < 5.0: 0 / 100800 (0.00%)
```

Per-attribute breakdown:

```
Attr 1: 14400 elems, Vp=6050, Vs=3543, PPW_min=6.3, dt_min=1.07e-03
Attr 2: 86400 elems, Vp=3066, Vs=2049, PPW_min=11.0, dt_min=2.82e-03
```

The fraction of PPW-deficient elements drops to 0%, and $PPW \geq 5$ is satisfied for every element.

5.5.5 Step 4: Running the simulation

Run the simulation with the refined mesh:

```
mpirun -np 64 ./build/src/semsws \
  --config config_visco_sim.yaml
```

5.6 Visualization scripts

Python visualization scripts are provided under `scripts/plot/`. All of them require `matplotlib` and `numpy`.

5.6.1 `plot_wavefield_2d.py` — 2D wavefield visualization

Visualizes 2D wavefield snapshots and material distributions output in the GMT format using `matplotlib`.

```
python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results
```

The following files are produced under `results/figures/`:

- `wavefield_snapshots.png` — Wavefield snapshots (all time steps)
- `material.png` — Material distribution

See Figure 3.1 in Chapter 3 for an example.

5.6.2 `plot_wavefield_3d.py` — 3D cross-section wavefield visualization

Visualizes 3D cross-sections (`cross_sections`) output in the GMT format using `matplotlib`.

```
python3 scripts/plot/plot_wavefield_3d.py --results-dir ./results
```

Snapshots are produced for each cross-section (`yz`, `xz`, `xy`) configured. Figure 5.2 shows an example output for the `xz` cross-section ($y = 5000$ m) of the two-layer viscoelastic model.

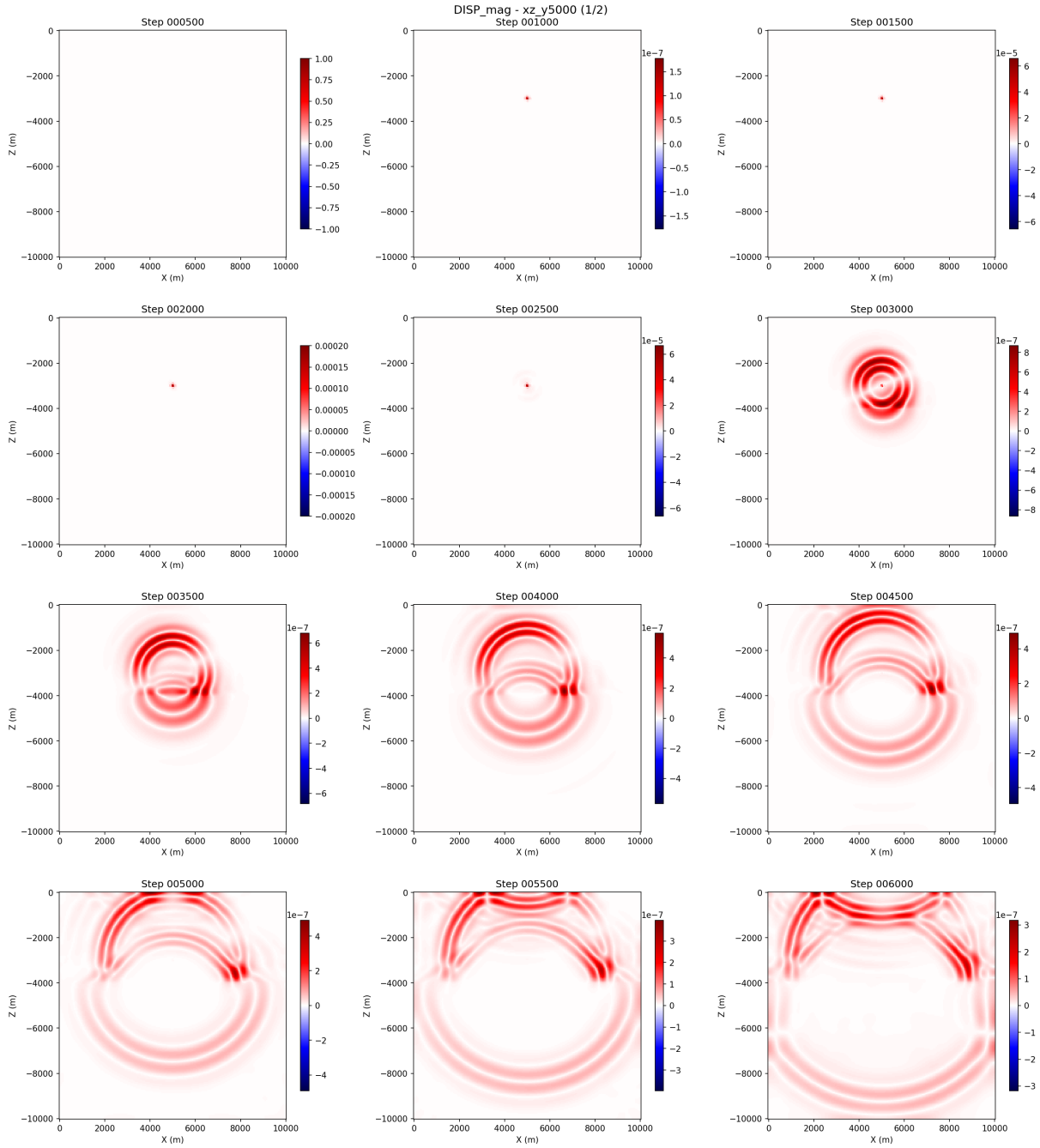


Figure 5.2: 3D cross-section snapshot produced by `plot_wavefield_3d.py` (xz cross-section, $y = 5000$ m). Displacement magnitude in the two-layer viscoelastic model. Output every 500 steps.

5.6.3 `plot_3d_slices.py` — PyVista 3D visualization

Uses PyVista to visualize three cross-sections placed in 3D space. The `pyvista` package is required additionally.

```
python3 scripts/plot/plot_3d_slices.py --results-dir ./results \
  --type wavefield --step 6000
```

Option	Description
<code>--results-dir</code>	Results directory
<code>--type</code>	<code>wavefield</code> or <code>material</code>
<code>--step</code>	Time step number to display (for wavefield)

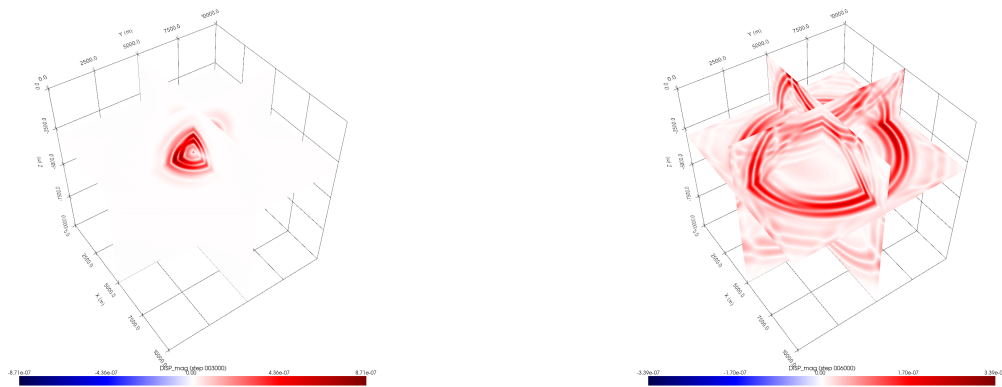


Figure 5.3: PyVista 3D visualization produced by `plot_3d_slices.py`. yz, xz, xy cross-sections are placed in 3D space. Left: Step 3000. Right: Step 6000. Displacement magnitude in the two-layer viscoelastic model.

5.6.4 `plot_mesh_quality.py` — Mesh quality histograms

Generates histogram figures from the CSV files output by `evaluate_mesh`.

```
# CSV output from evaluate_mesh
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist

# Generate the histogram figure
python3 scripts/plot/plot_mesh_quality.py mesh_hist
python3 scripts/plot/plot_mesh_quality.py mesh_hist --output quality.png
```

Histograms of element size, CFL $\Delta t_{\max}$, PPW, aspect ratio, and Jacobian determinant are gathered into a single figure.

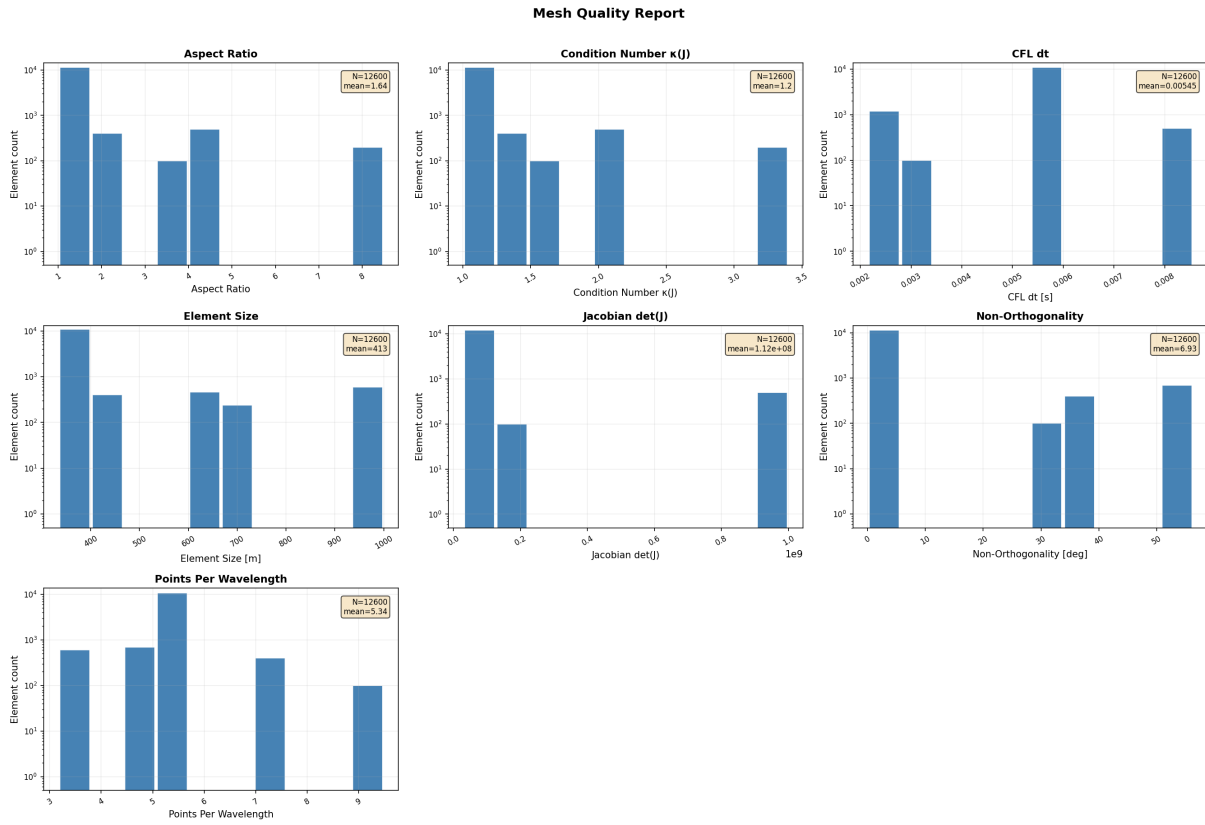


Figure 5.4: Mesh quality histograms (the initial mesh before refinement). The output of `evaluate_mesh` visualized with `plot_mesh_quality.py`.

Chapter 6

Application Examples

The `examples/` directory contains sample cases that demonstrate the features of SEMSW. This chapter introduces the main examples excluding the FWI-related ones. Each example comes with a README and a run script, so you can follow the procedure as documented.

6.1 visualization — Visualization demos

`examples/visualization/` contains demonstrations that exercise all output formats (GLVis, ParaView, GMT) on 2D/3D acoustic and elastic wave simulations.

Directory	Model	Description
2D/acoustic/	2D acoustic	Used in Chapter 3
2D/elastic/	2D elastic	Force source
3D/acoustic/	3D acoustic	3D cross-section output (<code>cross_sections</code>)
3D/elastic/	3D elastic	Moment-tensor source

Each example can be run with `run.sh`:

```
cd examples/visualization/3D/elastic
bash run.sh 4      # argument is the MPI process count
```

After the simulation, `run.sh` automatically performs visualization with `plot_wavefield_2d.py` (2D) or `plot_wavefield_3d.py` (3D).

The 3D examples set cross-section positions through `cross_sections` in the config, and Figures 5.2 and 5.3 in Chapter 5 were generated from the same setup as the 3D elastic example.

6.2 elastic_analytic — Analytical-solution benchmark

`examples/elastic_analytic/` is a benchmark that compares the numerical solution against the analytical solution by Aki & Richards (2002) for a double-couple source in a homogeneous isotropic elastic medium.

6.2.1 Problem setup

Parameter	Value
Domain	44 km $\times$ 44 km $\times$ 34 km
Mesh	Internally generated, 120 $\times$ 120 $\times$ 100 elements
Polynomial order	4
Material	$V_p = 2800$ m/s, $V_s = 1500$ m/s, $\rho = 2300$ kg/m ³
Source	Double couple (M_{rp} component), at the domain center
Receivers	6 points (Z1–Z6) at varying distances from the source

6.2.2 How to run

```
cd examples/elastic_analytic
bash run_benchmark.sh 4      # argument is the MPI process count
```

The script automates three steps:

1. Generate the analytical solution with `generate_analytical.py` (`ref/*.semd`).
2. Run the simulation with `semsws`.
3. Compute and display the normalized L2 errors of each trace (6 stations $\times$ 3 components = 18 traces).

6.2.3 Visualization of the result

```
python3 plot_comparison.py
```

A waveform-comparison figure with 3 stations $\times$ 3 components is produced. The analytical solution (solid black), the numerical solution (dashed red), and the residual scaled by 10 (blue) are overlaid.

6.3 visco_elastic_3d_layered — 3D viscoelastic two-layer model

`examples/visco_elastic_3d_layered/` is an example of a 3D two-layer model using an external mesh generated by CUBIT. The workflow for mesh evaluation and refinement is described in detail in §5.5 of Chapter 5.

6.3.1 Included configurations

File	Description
<code>config_elastic.yaml</code>	Elastic (no attenuation), reads external mesh directly
<code>config_elastic_sim.yaml</code>	Elastic, uses the refined partitioned mesh
<code>config_visco.yaml</code>	Viscoelastic (with attenuation), reads external mesh directly
<code>config_visco_sim.yaml</code>	Viscoelastic, uses the refined partitioned mesh
<code>mesh_layered_3d.jou</code>	CUBIT journal file
<code>materials_elastic.txt</code>	Elastic properties ($Q=9999$, effectively no attenuation)
<code>materials_visco.txt</code>	Viscoelastic properties ($Q_\kappa = Q_\mu = 50\text{--}100$)

6.3.2 How to run


```
cd examples/visco_elastic_3d_layered

# 1. Generate the mesh with CUBIT
coreform_cubit -nographics -input mesh_layered_3d.jou

# 2. Refine the mesh (64 processes)
mpirun -np 64 ../../build/src/refine_mesh \
  --config config_visco.yaml --output-dir mesh/

# 3. Run the simulation
mpirun -np 64 ../../build/src/semsws \
  --config config_visco_sim.yaml
```

Or run everything at once with the bundled script:

```
bash run_visco.sh
```

A script for comparing elastic and viscoelastic waveforms is also provided:

```
python3 plot_comparison.py
```

6.4 fun

`examples/fun/` contains examples that use unstructured meshes generated by the HOHQMesh library. They were made casually as a break from coding, so the configuration may have issues. HOHQMesh is an open-source high-order hexahedral/tetrahedral mesh generator that allows the creation of complex-shape meshes through text-based input files.

These examples serve as practical illustrations of using a mesh source other than CUBIT/GMSH.

6.4.1 ICM — 2D elastic wave

`examples/fun/icm/` is a 2D elastic wave simulation on a mesh shaped like the letters “ICM”. A source is placed at the top dot of “i”. After running a forward simulation that shows the wave spreading, reordering the snapshots backwards generates an animation in which the wave converges back to the dot of “i”.

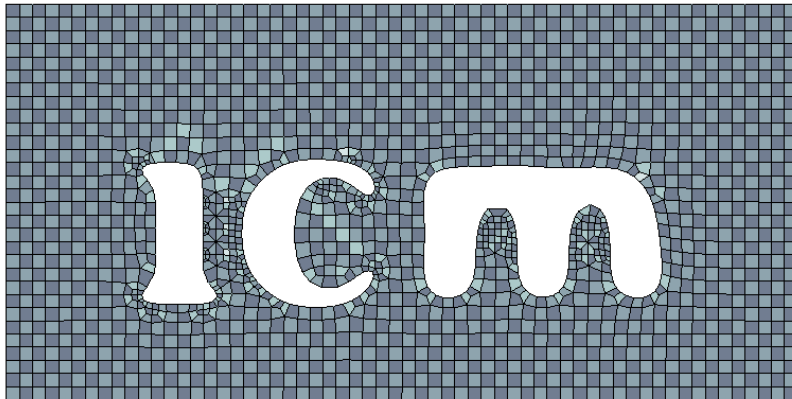


Figure 6.1: The “ICM” letter-shape mesh generated by HOHQMESH.

```
cd examples/fun/icm
bash run.sh
```

File	Description
<code>icm.control</code>	HOHQMESH control file
<code>generate_mesh.py</code>	Convert the HOHQMESH output to the GMSH format
<code>config.yaml</code>	Simulation configuration (2D elastic, order 8)
<code>make_animation.py</code>	Generate the animation from the snapshots

Demo video (reverse-played animation): https://youtu.be/Qts2n_2wtfI

6.4.2 JAMSTEC — 2D acoustic wave

`examples/fun/jamstec/` is a 2D acoustic wave simulation on a mesh shaped like the letters “JAMSTEC”. As with ICM, a source is placed at the top dot of “J”, and the reverse playback produces an animation in which the wave converges.



Figure 6.2: The “JAMSTEC” letter-shape mesh generated by HOHQMesh.

```
cd examples/fun/jamstec  
bash run.sh
```

Demo video (reverse-played animation): https://youtu.be/_yijXLc3F6U

These examples read the HOHQMesh output (.msh) as `mesh.type: external`. The same approach can be used to run simulations on meshes of arbitrary shapes.

Chapter 7

Performance

This chapter presents the parallel performance of SEMSWS on the Earth Simulator (ES, JAMSTEC) and the LUMI supercomputer (CSC, Finland). These results are taken from the paper.

7.1 Test environment

Table 7.1: Systems used for testing

System	Partition	Configuration
ES (JAMSTEC)	CPU	AMD EPYC 7742 (128 cores/node), 720 nodes
ES (JAMSTEC)	GPU	NVIDIA A100 (8 GPUs/node), 8 nodes
LUMI (CSC)	GPU	AMD MI250X (4 modules = 8 GCDs/node), 2978 nodes

All tests run the 3D isotropic elastic wave equation with order-4 polynomials (5 GLL points) for 1000 time steps. The mesh is a structured hexahedral mesh with Cartesian partitioning, and GPU runs use GPU-aware MPI.

7.2 Strong Scaling

In the strong scaling test, the problem size is fixed at 5.24M elements (337.3M mesh nodes) while the process count is increased.

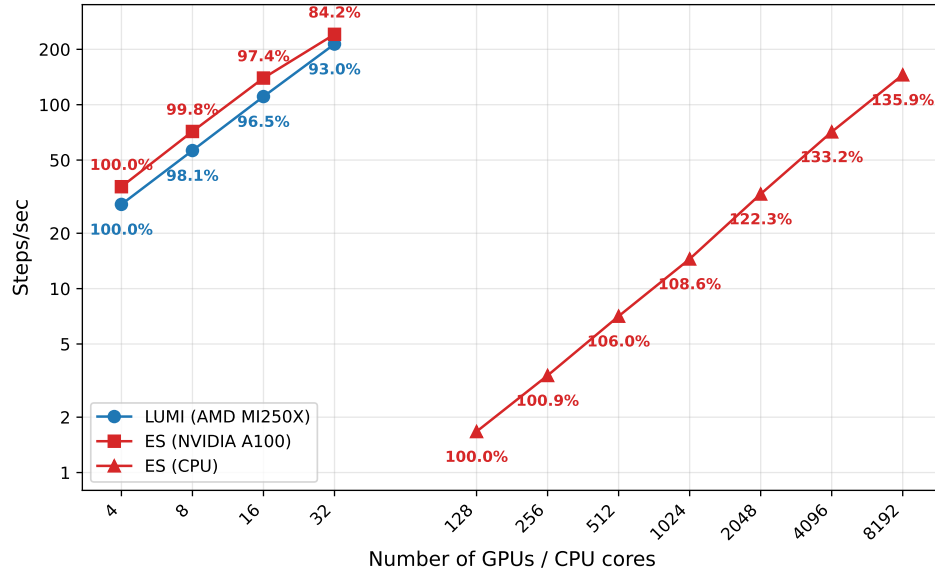


Figure 7.1: Strong scaling results. ES CPU nodes (AMD EPYC 7742), ES GPU nodes (NVIDIA A100), LUMI GPU nodes (AMD MI250X). The number next to each symbol is the parallel efficiency (%). The baselines are 128 cores for CPU and 4 units for GPU.

On ES CPU, super-linear scaling (parallel efficiency 135.9%) was observed up to 8192 cores. This is likely due to improved cache efficiency as the per-core problem size decreases. For GPUs, parallel efficiency of 84.2% (ES) and 93.0% (LUMI) was maintained up to 32 GPUs.

7.3 Weak Scaling

In the weak scaling test, the per-CPU problem size is fixed at 15,210 elements and the per-GPU size at 1,600,000 elements, while the total problem size grows proportionally with the process count.

Table 7.2: Problem sizes in the weak scaling test

CPU (ES, 15,210 elements/core)				GPU (ES & LUMI, 1,600,000 elements/GPU)			
nCPU	Elements	Mesh nodes	DoFs	nGPU	Elements	Mesh nodes	DoFs
128	1.95M	125.4M	376.2M	4	6.40M	412.7M	1.24B
512	7.79M	500.8M	1.50B	8	12.8M	825.1M	2.48B
2048	31.2M	2.00B	6.00B	16	25.6M	1.65B	4.95B
8192	124.6M	8.00B	24.0B	32	51.2M	3.30B	9.90B
				GPU (LUMI only)			
				320	512.0M	32.8B	98.4B
				640	1.02B	65.6B	196.8B
				1280	2.05B	131.2B	393.6B
				2560	4.10B	262.4B	787.2B
				5120	8.19B	524.8B	1.57T
				8192	13.1B	839.6B	2.52T

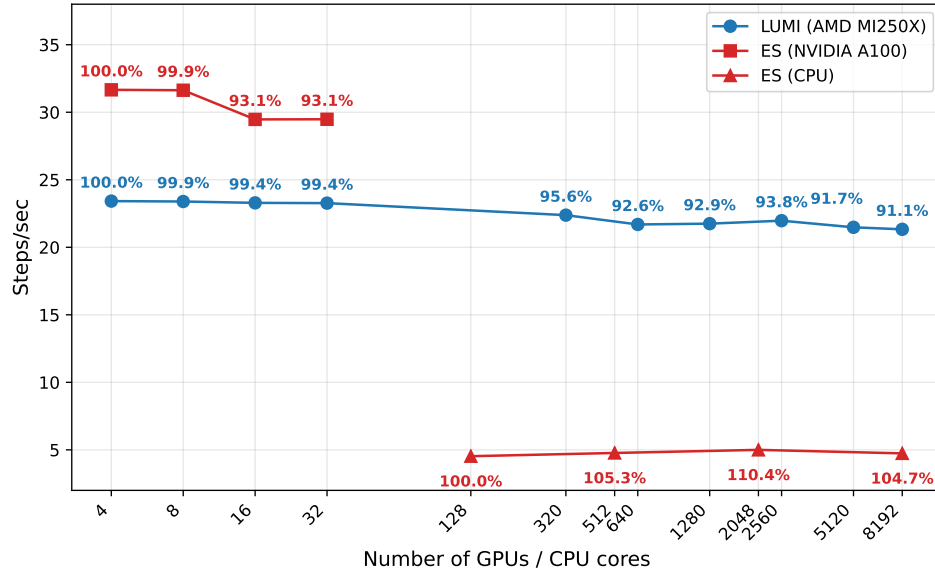


Figure 7.2: Weak scaling results. ES CPU nodes, ES GPU nodes, LUMI GPU nodes. The baselines are 128 cores for CPU and 4 units for GPU. Note that the per-process problem size differs between CPU and GPU.

On CPUs, parallel efficiency of 100% or more was maintained up to 8192 cores. On GPUs, an efficiency of 93% or more was achieved up to 32 NVIDIA A100 GPUs on ES, and 91% or more up to 8192 AMD MI250X GPUs on LUMI. The largest configuration (LUMI 8192 GPUs) reached a computation of 839.6 billion mesh nodes (2.52 trillion DoFs).

7.4 GPU vs CPU

Using the strong scaling results on ES, this section compares the performance of 1 GPU node (8 NVIDIA A100s) against N CPU nodes (128 cores/node).

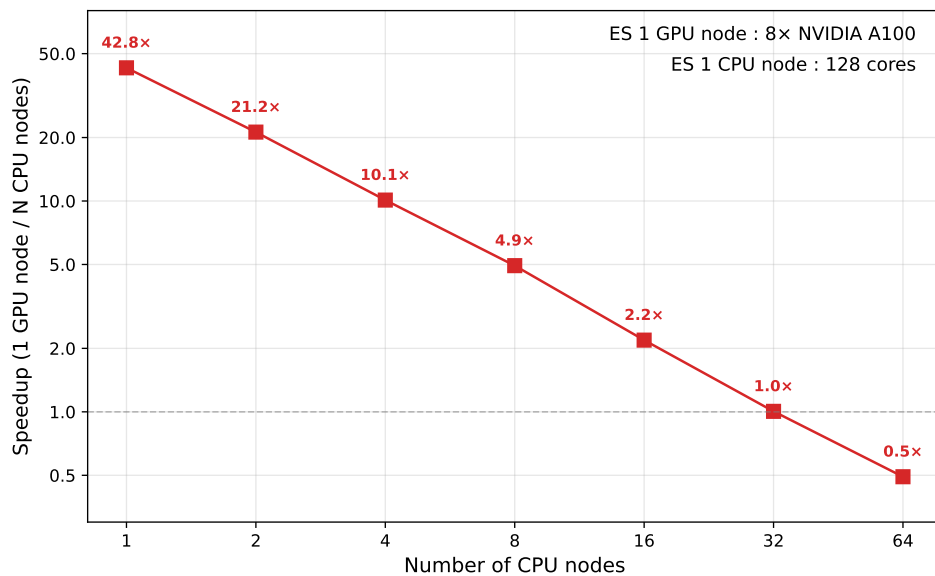


Figure 7.3: Speedup of 1 GPU node (8 NVIDIA A100s) over N CPU nodes (128 cores/node). Problem size: 5.24M elements.

A single GPU node achieves a $42.8\times$ speedup over a single CPU node. A single GPU node delivers performance equivalent to 32 CPU nodes (4,096 cores).

Development Team

- Kota Mukumoto — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain
- Yann Capdeville — Nantes Université, Univ Angers, Le Mans Université, CNRS, Laboratoire de Planétologie et Géosciences, Nantes, France
- Kazuya Shiraishi — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Ryoichiro Agata — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Gou Fujie — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Thomas Bodin — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain